

SUPERVISED LEARNING

+ Definitions

+ LINEAR Regression

+ Batch & stochastic gradient descent

+ Normal Equations

Supervised learning

Prediction

$$h: \mathcal{X} \rightarrow \mathcal{Y}$$

Image

Contains car

TEXT

IS HAVE SCREEN?

House Data

PRICE

Given: Training Set $\rightarrow$

$$\{(x^{(1)}, y^{(1)}) \dots (x^{(n)}, y^{(n)})\} \quad x^{(i)} \in \mathcal{X}, y^{(i)} \in \mathcal{Y}$$

Do: find "good" $h: \mathcal{X} \rightarrow \mathcal{Y}$ hypothesis

this job of training algorithm

WE USE h ON NEW DATA (x)

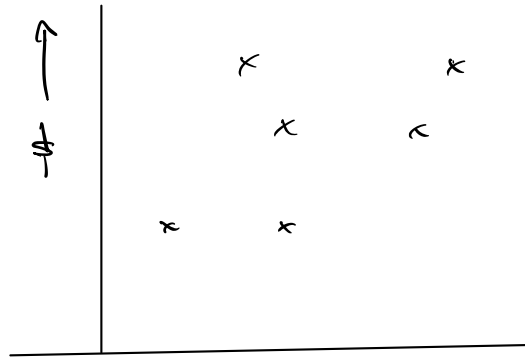
call this Prediction, WE ARE VERY INTERESTED IN $x \notin$ TRAINING SET

if y IS DISCRETE $\Rightarrow$ classification

y IS CONTINUOUS $\Rightarrow$ Regression

Example Data (House Prices)

SQ ft	Price (k)
2100	400
2500	900
1127	800
⋮	⋮



TRAINING SET

Learning Algo

SQ ft.

$h: \text{SQ ft} \rightarrow \text{Price}$

How do we represent h^2 ?

$$h(x) = \theta_0 + \theta_1 x_1 \quad (\text{affine fn.})$$

	x_1 Size	x_2 Bedrooms	x_3 Lot size	Price
$x^{(1)}$	2104	4 $x_2^{(1)}$	45K	400
$x^{(2)}$	2500	3	30K	900

$$h(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots$$

$$= \sum_{j=0}^3 \theta_j x_j \quad \text{NB } x_0 \text{ identically 1}$$

$$\theta = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \theta_2 \\ \theta_3 \end{bmatrix} \quad x^{(i)} = \begin{bmatrix} x_0^{(i)} \\ x_1^{(i)} \\ x_2^{(i)} \\ x_3^{(i)} \end{bmatrix} \quad \begin{matrix} 1 \\ \text{SIZE} \\ \text{BEDROOMS} \\ \text{LOT SIZE} \end{matrix}$$

$y^{(i)}$ is Price

PARAMETERS

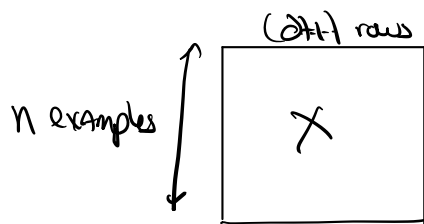
INPUTS / FEATURES

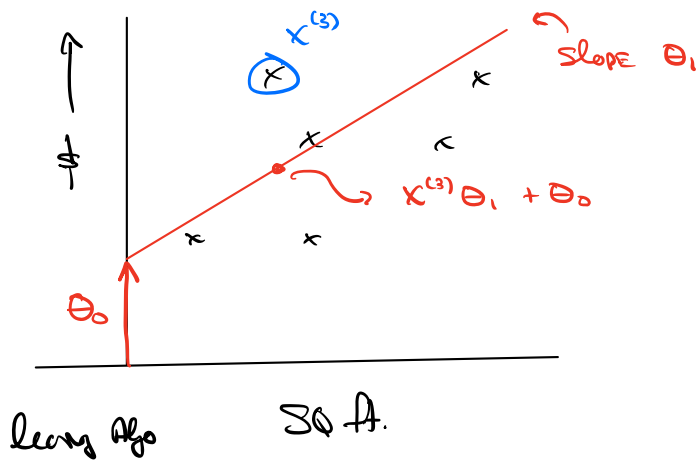
Output / Target

(x, y) is a training example

$(x^{(i)}, y^{(i)})$ is i^{th} example i runs $1 \dots n$

n examples and d features $\Rightarrow x_i^{(i)} \theta$ are $d+1$ dimensional





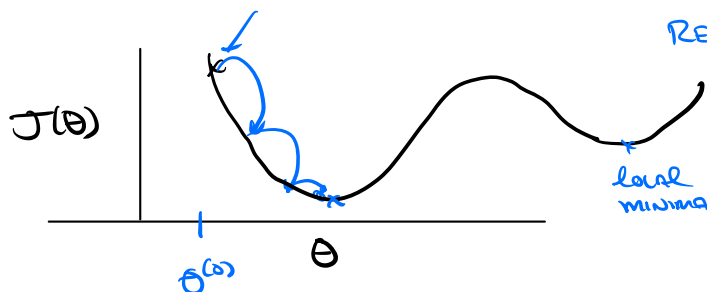
$$h_{\theta}(x) = \sum_{j=0}^2 \theta_j x_j \quad \text{WANT TO CHOOSE } \theta \text{ st. } h_{\theta}(x) \approx y$$

IDEA: $J(\theta) = \frac{1}{2} \sum_{i=1}^n (h_{\theta}(x^{(i)}) - y^{(i)})^2$ Cost function (least squares)

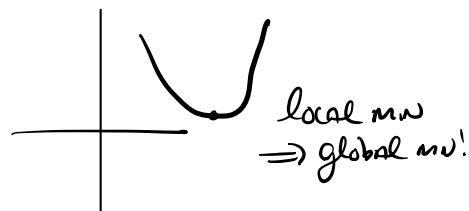
$$\min_{\theta} J(\theta)$$

Gradient Descent

START $\theta^{(0)}$ AT RANDOM OR ZERO



if J is nice (convex)



$$\theta^{(0)} := 0$$

$$\theta_j^{(t+1)} := \theta_j^{(t)} - \alpha \frac{\partial J(\theta^{(t)})}{\partial \theta_j}$$

LEARNING RATE

$$j = 1 \dots d$$

$$\frac{\partial J(\theta)}{\partial \theta_j} = \sum_{i=1}^n \frac{\partial}{\partial \theta_j} \left(\frac{1}{2} (h_{\theta}(x^{(i)}) - y^{(i)})^2 \right)$$

$$= \sum_{i=1}^n (h_{\theta}(x^{(i)}) - y^{(i)}) \frac{\partial}{\partial \theta_j} h_{\theta}(x^{(i)})$$

$$h_{\theta}(x^{(i)}) = \theta_0 x_0^{(i)} + \theta_1 x_1^{(i)} + \dots + \theta_d x_d^{(i)}$$

$$\frac{\partial h_{\theta}(x^{(i)})}{\partial \theta_j} = x_j^{(i)}$$

$$\theta_j^{(t+1)} := \theta_j^{(t)} - \alpha \sum_{i=1}^n (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

SOMETIMES WRITE AS $\theta^{(t+1)} := \theta^{(t)} - \alpha \sum_{i=1}^n (h_{\theta}(x^{(i)}) - y^{(i)}) x^{(i)}$

vector notation

BATCH VERSUS STOCHASTIC MINIBATCH

$$\Theta^{(t+1)} := \Theta^{(t)} - \alpha \sum_{i=1}^n (h_{\Theta}(x^{(i)}) - y^{(i)}) x^{(i)}$$

Minibatch: Randomly select $b < n$ points AND estimate gradient

1. Pick b points $\{i_1, \dots, i_b\} = B$

2.

$$\Theta^{(t+1)} := \Theta^{(t)} - \alpha_b \sum_{k \in B} (h_{\Theta}(x^{(k)}) - y^{(k)}) x^{(k)}$$

One detail Scale α and α_b differently.

Tradeoff: Noisier BUT much faster

faster: Imagine if training set contains 100 copies of same point

$\Rightarrow$ Not as ridiculous as it seems (near copies)

How do you choose B ? Surely, whatever works

Normal Equation

$$\nabla_{\theta} J(\theta) = \begin{bmatrix} \frac{\partial}{\partial \theta_0} J(\theta) \\ \frac{\partial}{\partial \theta_1} J(\theta) \\ \vdots \end{bmatrix}$$

$$A \in \mathbb{R}^{2 \times 2}$$

$$A = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix}$$

$$f: A \rightarrow \mathbb{R}$$

then

$$\nabla_A f(A) = \begin{bmatrix} \frac{\partial f}{\partial a_{11}} & \frac{\partial f}{\partial a_{12}} \\ \frac{\partial f}{\partial a_{21}} & \frac{\partial f}{\partial a_{22}} \end{bmatrix}$$

Now, we want to find minimum

$$\nabla_{\theta} J(\theta) = \vec{0} \quad (\nabla_{\theta} J(\theta) \in \mathbb{R}^{d+1})$$

$$J(\theta) = \frac{1}{2} \sum_{i=1}^n (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

$$X = \begin{bmatrix} -x^{(1)} \\ x^{(2)} \\ \vdots \\ x^{(n)} \end{bmatrix} \in \mathbb{R}^{n \times d} \quad \text{Design matrix}$$

$$X\theta = \begin{bmatrix} -x^{(1)} \\ \vdots \\ x^{(n)} \end{bmatrix} \begin{bmatrix} \theta_0 \\ \theta_1 \\ \vdots \end{bmatrix} = \begin{bmatrix} h_{\theta}(x^{(1)}) \\ \vdots \\ h_{\theta}(x^{(n)}) \end{bmatrix}$$

$$y = \begin{bmatrix} y^{(1)} \\ \vdots \\ y^{(n)} \end{bmatrix}$$

$$\text{then } J(\theta) = \frac{1}{2} (X\theta - y)^T (X\theta - y)$$

$$\nabla_{\theta} J(\theta) = X^T X \theta - X^T y = 0 \Rightarrow \theta = (X^T X)^{-1} X^T y$$

Optimal value.

CLASSIFICATION $\neq$ REGRESSION

PROBABILISTIC View of LINEAR REGRESSION

CLASSIFICATION

Why not LINEAR REGRESSION?

Logistic Regression

METHOD: Newton's METHOD

Recall Least Squares

Given $\{x^{(i)}, y^{(i)}\}$ for $i=1 \dots n$

in which $x^{(i)} \in \mathbb{R}^{d+1}$, $y^{(i)} \in \mathbb{R}$

Do find $\theta \in \mathbb{R}^{d+1}$ s.t. $\theta = \underset{\theta}{\operatorname{argmin}} \sum_{i=1}^n (y^{(i)} - h_{\theta}(x^{(i)}))^2$

where $h_{\theta}(x) = \theta^T x$

Why?

Assume $y^{(i)} = \theta_*^T x^{(i)} + \epsilon^{(i)}$

$\hookrightarrow$ Error or noise term.

Properties

1. $\mathbb{E}[\epsilon^{(i)}] = 0$ - IT'S UNBIASED

for $i \neq j$

2. THE ERRORS INDEPENDENT $\mathbb{E}[\epsilon^{(i)} \epsilon^{(j)}] = \mathbb{E}[\epsilon^{(i)}] \mathbb{E}[\epsilon^{(j)}]$ for $i \neq j$

How "Noisy" $\mathbb{E}[(\epsilon^{(i)})^2] = \sigma^2$

GAUSSIAN or Normal Distribution

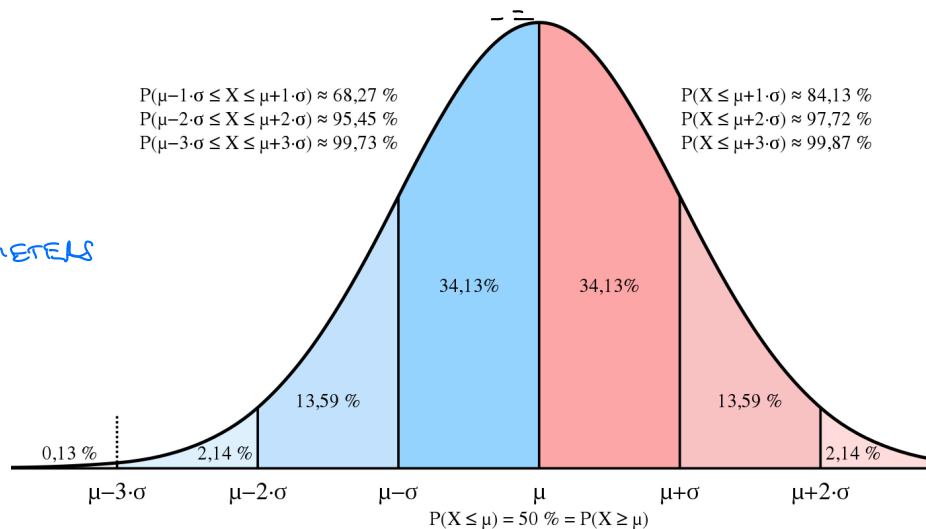
(UNIQUE of the ABOVE)

WRITE $E^{(i)} \sim N(\mu, \sigma^2)$
 $\mu = 0$ (MEAN)
 σ^2 (VARIANCE)

$$P(z; \mu, \sigma^2) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left\{-\frac{(z-\mu)^2}{2\sigma^2}\right\}$$

PARAMETERS

$$\exp(x) = e^x$$



$$P(y^{(i)} | x^{(i)}; \theta) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left\{-\frac{(y^{(i)} - \theta \cdot x^{(i)})^2}{2\sigma^2}\right\}$$

PARAMETER

$$y^{(i)} | x^{(i)}; \theta \sim N(\theta^T x^{(i)}, \sigma^2)$$

Picking $\theta \Rightarrow$ Picks distribution

Likelihood among many distributions, "most likely"

$$\mathcal{L}(\theta) = P(y | x; \theta) = \prod_{i=1}^n P(y^{(i)} | x^{(i)}; \theta) \text{ (iid)}$$

$$= \prod_{i=1}^n \frac{1}{\sigma\sqrt{2\pi}} \cdot \exp\left\{-\frac{(y^{(i)} - \theta \cdot x^{(i)})^2}{2\sigma^2}\right\}$$

log likelihood

$$\mathcal{L}(\theta) = \log \mathcal{L}(\theta) = \sum_{i=1}^n \left(\log \frac{1}{\sigma\sqrt{2\pi}} - \frac{(y^{(i)} - \theta \cdot x^{(i)})^2}{2\sigma^2} \right)$$

Requires only one θ .

$$\mathcal{J}(\theta) = \max_{\theta} \mathcal{L}(\theta) = \min_{\theta} \frac{1}{2} \sum_{i=1}^n (y^{(i)} - \theta \cdot x^{(i)})^2$$

SAME RECIPE AS linear Regression!

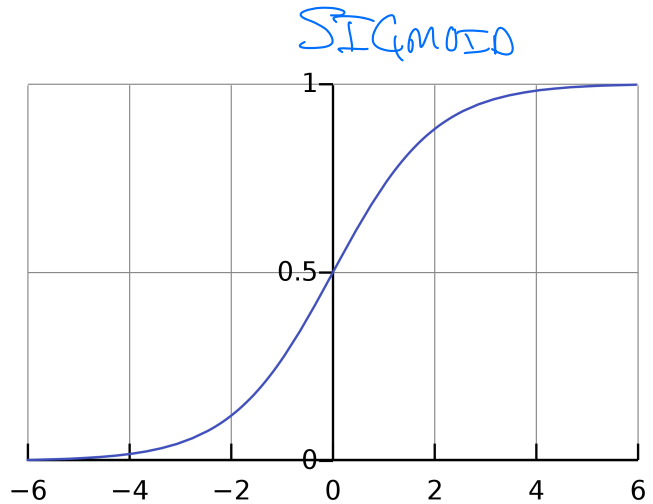
$$h_{\theta}(x) \in [0, 1]$$

$$h_{\theta}(x) = g(\theta^T x) = (1 + e^{-\theta^T x})^{-1}$$

$$g(z) = \frac{1}{1 + e^{-z}} \text{ "link function"}$$

$$P(y=1 | x; \theta) = h_{\theta}(x)$$

$$P(y=0 | x; \theta) = 1 - h_{\theta}(x)$$



likelihood

$$\mathcal{L}(\theta) = P(\vec{y} | \vec{X}; \theta) = \prod_{i=1}^n P(y^{(i)} | x^{(i)}; \theta)$$

log likelihood

$$\log \mathcal{L}(\theta) = \sum_{i=1}^n \log h_{\theta}(x^{(i)})^{y^{(i)}} (1 - h_{\theta}(x^{(i)}))^{1-y^{(i)}}$$

$$\log \mathcal{L}(\theta) = \sum_{i=1}^n y^{(i)} \log h_{\theta}(x^{(i)}) + (1 - y^{(i)}) \log (1 - h_{\theta}(x^{(i)}))$$

SAME RECIPE:

$$\theta^{(+1)} = \theta^{(+)} + \alpha \frac{\partial}{\partial \theta_i} \mathcal{J}(\theta)$$

$$\frac{\partial \mathcal{L}(\theta)}{\partial \theta_j} = \sum_{i=1}^n (y^{(i)} - h_{\theta}(x^{(i)})) x_j^{(i)}$$

Rule is very general.

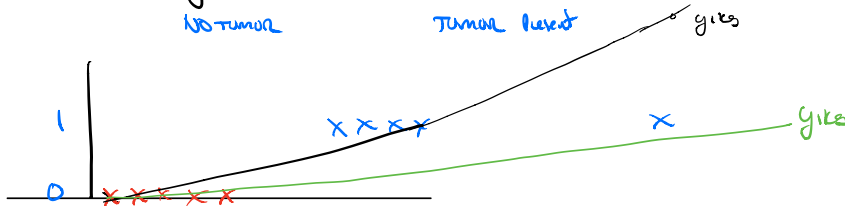
Classification

Given $\{(x^{(i)}, y^{(i)}) \text{ for } i=1 \dots n\}$

$$y^{(i)} \in \{0, 1\}$$

negative class
no tumor

Positive class
tumor present

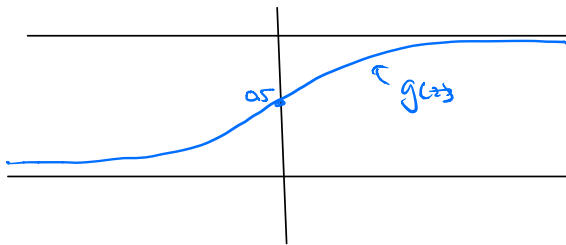


SAME RECIPE

Want $h_\theta(x) \in [0, 1]$

$$h_\theta(x) = g(\theta^T x) = \frac{1}{1 + e^{-\theta^T x}}$$

$$g(z) = \frac{1}{1 + e^{-z}} \quad \text{"link function"}$$



Sigmoid or a logistic function.

$$P(y=1 | x; \theta) = h_\theta(x)$$

$$P(y=0 | x; \theta) = 1 - h_\theta(x)$$

$$\begin{aligned} \mathcal{L}(\theta) &= P(\vec{y} | \vec{x}; \theta) = \prod_{i=1}^n P(y^{(i)} | x^{(i)}; \theta) \\ &= \prod_{i=1}^n h_\theta(x^{(i)})^{y^{(i)}} (1 - h_\theta(x^{(i)}))^{(1-y^{(i)})} \end{aligned} \quad \text{"ENCODS } \theta \text{"}$$

$$\ell(\theta) = \log \mathcal{L}(\theta) = \sum_{i=1}^n y^{(i)} \log h_\theta(x^{(i)}) + (1 - y^{(i)}) \log (1 - h_\theta(x^{(i)}))$$

SAME RECIPE MAXIMIZE with gradient ascent

$$\theta_1 := \theta_1 + \alpha \frac{\partial \ell(\theta)}{\partial \theta_1} \quad (\text{Gradient Ascent})$$

$$\theta \quad \theta \quad \theta_0 \quad \dots \quad \dots$$

of last week $\theta_j := \theta_0 - \alpha \frac{\partial}{\partial \theta_j} J(\theta)$ (least squares)

for OBSERVATION

$$\frac{\partial \ell(\theta)}{\partial \theta_j} = (y^{(i)} - h_{\theta}(x^{(i)})) x_j^{(i)} \quad \text{so } \dots$$

$$\theta := \theta - (h_{\theta}(x) - y^{(i)}) x^{(i)}$$

We'll SEE LATER this rule is very general.

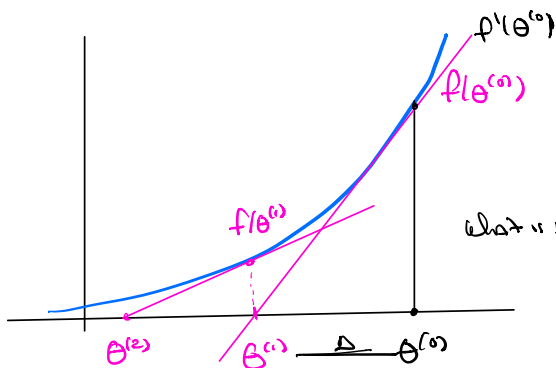
Newton's Method

Given $f: \mathbb{R}^D \rightarrow \mathbb{R}$

Do find x s.t. $f(x) = 0$.

ASIDE

$\max_{\theta} \ell(\theta)$ want $\ell'(\theta) = 0$
 $\uparrow$
 DERIVATIVE



$$\theta^{(n)} = \theta^{(n)} - \Delta$$

$$\text{What is } \Delta? \quad f(\theta^{(n)}) = f'(\theta^{(n)}) \cdot \Delta$$

$$\therefore \Delta = f'(\theta^{(n)})^{-1} f(\theta^{(n)})$$

$$\theta^{(n+1)} = \theta^{(n)} - \frac{f(\theta^{(n)})}{f'(\theta^{(n)})}$$

Converges fast! Quadratic $0.1 \rightarrow 0.01 \rightarrow 0.0001$ (digits double!)

Generalizing to vector $\theta \in \mathbb{R}^{D+1}$ $\ell'(\theta) = f(\theta)$ (i.e. minimization)

$$\theta^{(n+1)} = \theta^{(n)} - H^{-1} \nabla_{\theta} \ell(\theta)$$

$\nwarrow$ Hessian $\in \mathbb{R}^{(D+1) \times (D+1)}$ $\searrow \in \mathbb{R}^{D+1}$

$$H_{ij} = \frac{\partial^2}{\partial \theta_i \partial \theta_j} \ell(\theta)$$

NB! NO STEPSIZE!

Rough Comparison

	<u>Per iteration</u>	<u>Compute</u>	<u>STEPS TO ERROR ϵ</u>
SGD	1 DATA point	$O(d)$	ϵ^{-2}
BATCH SGD	N DATA points	$O(nd)$	$\approx \epsilon^{-1}$
Newton	N DATA points	$\Omega(nd^2)$	$\log(\frac{1}{\epsilon})$

IN CLASSICAL STATS d is small 100 or so

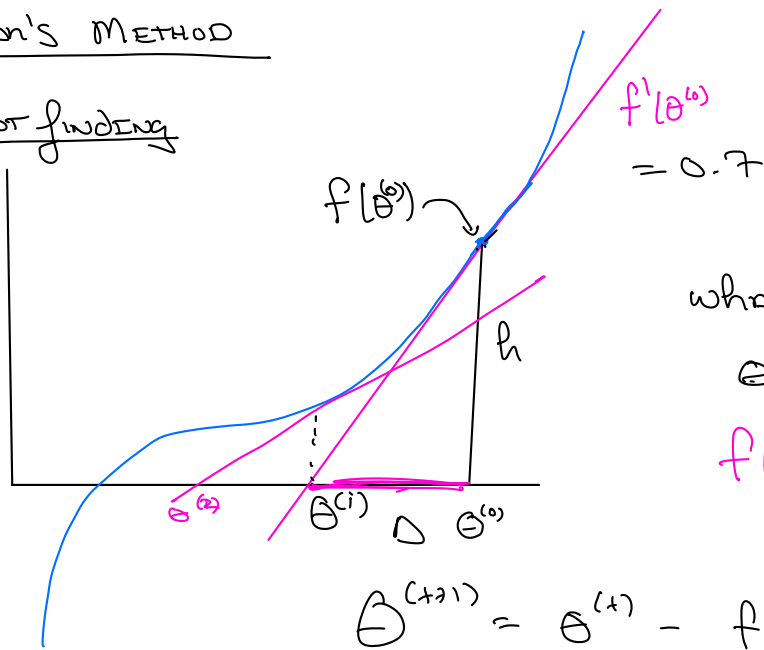
AND EXACT ANSWER MATTERS $\Rightarrow$ Newton (LBFGS)

MODERN ML d IS HUGE 175B $d^2 \Rightarrow \infty$

~~⇒~~ SGD of ~~ten~~ WORKHOUSE (Exact solution class is)

Newton's METHOD

Root finding



GIVEN $f: \mathbb{R}^d \rightarrow \mathbb{R}$

DO find $f(x) = 0$

(ASIDE $\text{know } l(\theta) \Rightarrow l'(\theta) = 0$)

what is Δ ?

$$\theta^{(1)} = \theta^{(0)} - \Delta$$

$$f(\theta^{(0)}) = f'(\theta^{(0)}) \cdot \Delta$$

$$\Delta = f'(\theta^{(0)})^{-1} \cdot f(\theta^{(0)})$$

$$\theta^{(k+1)} = \theta^{(k)} - \frac{f(\theta^{(k)})}{f'(\theta^{(k)})}$$

Leave 0.1 $\rightarrow$ 0.01 $\rightarrow$ 0.0001

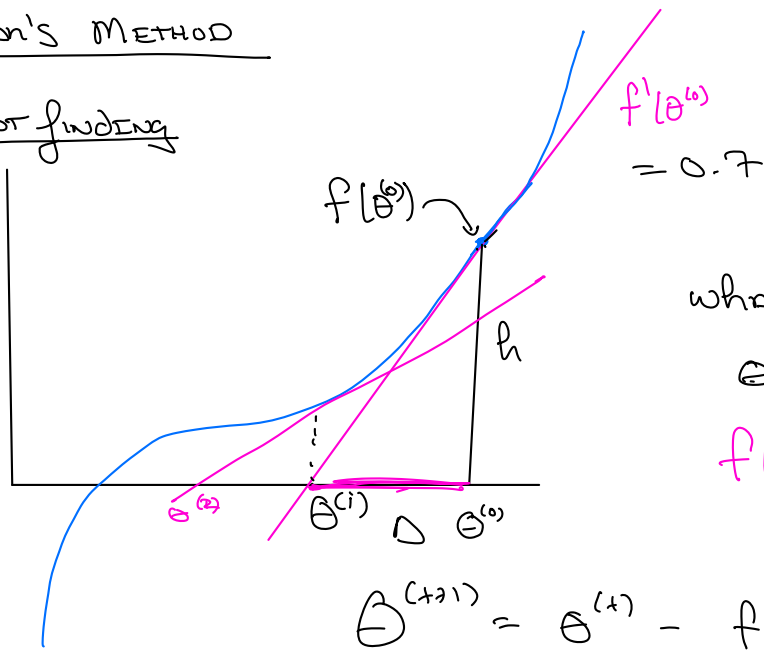
Generalize ∇ use for min. $\theta \in \mathbb{R}^{d+1}$ $l'(\theta) = f(\theta)$

$$\theta^{(k+1)} = \theta^{(k)} - \underbrace{H^{-1}}_{\text{Hessian} \in \mathbb{R}^{(d+1) \times (d+1)}} \underbrace{\nabla_{\theta} l(\theta)}_{\in \mathbb{R}^d}$$

$$H_{ij} = \frac{\partial^2}{\partial \theta_i \partial \theta_j} l(\theta)$$

Newton's METHOD

Root finding



GIVEN $f: \mathbb{R}^d \rightarrow \mathbb{R}$

DO find $f(x) = 0$

(ASIDE $\text{know } l(\theta) \Rightarrow l'(\theta) = 0$)

what is Δ ?

$$\theta^{(1)} = \theta^{(0)} - \Delta$$

$$f(\theta^{(0)}) = f'(\theta^{(0)}) \cdot \Delta$$

$$\Delta = f'(\theta^{(0)})^{-1} \cdot f(\theta^{(0)})$$

$$\theta^{(k+1)} = \theta^{(k)} - \frac{f(\theta^{(k)})}{f'(\theta^{(k)})}$$

Leave 0.1 $\rightarrow$ 0.01 $\rightarrow$ 0.0001

Generalize ∇ use for min. $\theta \in \mathbb{R}^{d+1}$ $l'(\theta) = f(\theta)$

$$\theta^{(k+1)} = \theta^{(k)} - H^{-1} \nabla_{\theta} l(\theta)$$

$\in \mathbb{R}^d$

$\hookrightarrow$ Hessian $\in \mathbb{R}^{(d+1) \times (d+1)}$

$$H_{ij} = \frac{\partial^2}{\partial \theta_i \partial \theta_j} l(\theta)$$

Rough Comparison

<u>METHOD</u>	<u>Per iteration</u>	<u>Compute</u>	<u>Steps to Error ϵ^{-1}</u>
SGD	1 Data Point	$\mathcal{O}(d)$	ϵ^{-2}
BATCH GD	N Data Points	$\mathcal{O}(nd)$	$\approx \epsilon^{-1}$
Newton Method	N Data points	$\Omega(nd^2)$	$\approx \log(\frac{1}{\epsilon})$

MINIBATCH

It's classical STATS d was small
 n was massive

Exponential family models

- Definition & motivation
- Examples
- Softmax (multiclass Classification)

Unify INFERENCE &
LEARNING for many
MODELS

Exponential family

PDF. IDEA "If p has special form $\Rightarrow$ some questions for free"

$$p(y; \eta) = b(y) \exp[\eta^T T(y) - a(\eta)]$$

DATA
NATURAL PARAMETERS

$T(y)$ is called sufficient statistic (we'll use $T(y) = y$ so)
is same dim as η

$b(y)$ is called base measure. Does not depend on η

$a(\eta)$ is called log partition function. Does not depend on y

$\Rightarrow$ IT MAKES SURE p IS probability function

$y, a(\eta), b(y)$ ARE SCALARS

$\eta, T(y)$ ARE SAME DIMENSION

Examples

Bernoulli ϕ is probability of an event

$$\begin{aligned} p(y; \phi) &= \phi^y (1-\phi)^{1-y} \\ &= \exp(y \log \phi + (1-y) \log (1-\phi)) \\ &= \exp\left(\log \frac{\phi}{1-\phi} \cdot y + \log (1-\phi)\right) \end{aligned}$$

CHECK fits into form:

$$p(y; \eta) = b(y) \exp[\eta^T \tau(y) - a(\eta)]$$

$$\tau(y) = y \quad \eta = \log \frac{\phi}{1-\phi} \quad b(y) = 1$$

Claim: $-a(\eta) = \log (1-\phi)$

OBSERVE: $\eta = \log \frac{\phi}{1-\phi} \Rightarrow \phi = \frac{e^\eta}{1+e^\eta}$

Here, $1-\phi = \frac{e^{-\eta}}{1+e^{-\eta}} = \frac{1}{1+e^\eta}$ so $-\log(1-\phi) = \log(1+e^\eta) \square$.

Example #2 Gaussian (μ) fixed variance) $\sigma^2 = 1$

$$p(y; \mu) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{(y-\mu)^2}{2}\right)$$

$$= \underbrace{\frac{1}{\sqrt{2\pi}} e^{-\frac{\mu^2}{2}}}_{b(\mu)} \exp(\mu y - \frac{1}{2} \mu^2)$$

$$p(y; \eta) = b(y) \exp[\eta^T \tau(y) - a(\eta)]$$

$$\eta = \mu \quad \tau(y) = y \quad \text{and} \quad a(\eta) = \frac{1}{2} \eta^2 \quad \checkmark$$



Why do we care about this form?

Inference is "easy"

$$E[y; \eta] = \frac{\partial}{\partial \eta} a(\eta)$$

$$\text{VAR}[y; \eta] = \frac{\partial^2}{\partial^2 \eta} a(\eta)$$

Learning is "well defined"

MLE w.r.t to η is CONCAVE

(so negative log likelihood is convex)

Generalized Linear Models (GLM)

Design choices $\Rightarrow$ Assumptions.

(i) $y|x; \theta \sim$ Exponential family

Binary $\rightarrow$ Bernoulli

Real $\rightarrow$ Gaussian

Counts $\rightarrow$ Poisson

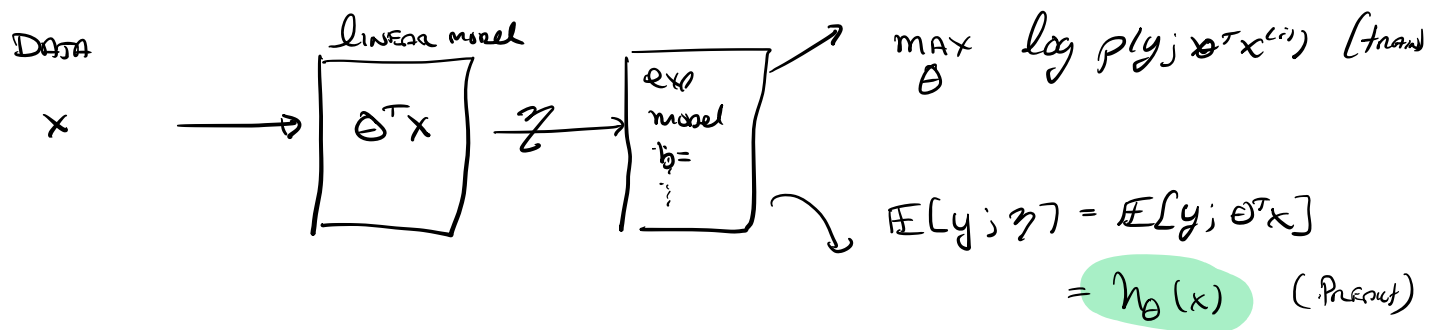
$\mathbb{R}^+$ $\rightarrow$ Gamma, exponential

Distribution $\rightarrow$ Dirichlet

(ii) $\eta = \theta^T x$ $\theta \in \mathbb{R}^d$, $x \in \mathbb{R}^d$

(iii) Define @ test time

Output $\mathbb{E}[y|x; \theta]$ i.e. $h_\theta = \mathbb{E}[y|x; \theta]$



learning $\theta_j := \theta_j + \alpha (y^{(i)} - h_\theta(x^{(i)})) x_j^{(i)}$

Terminology

Model parameter

θ

train on $\mathcal{D}_{\text{tr}}$

$$\theta^T x \rightarrow$$

Natural Parameter

η

$$\begin{array}{c} \xrightarrow{g} \\ \xleftarrow{g^{-1}} \end{array}$$

Canonical

ϕ : Bernoulli

μ, σ^2 : Gaussian

λ : Poisson

g is called the canonical response function
 g^{-1} " " the link function

$$\mu = \mathbb{E}[y | \eta] \triangleq g(\eta)$$

$$\Rightarrow \frac{\partial \ln \pi}{\partial \eta} = g(\eta)$$

logistic regression (Bernoulli)

$$h_{\theta}(x) = \mathbb{E}[y | x; \theta] = \phi = \overset{\text{Canonical}}{1} \overset{\text{Natural}}{=} \frac{1}{1 + e^{-\eta}} = \overset{\text{model}}{=} \frac{1}{1 + e^{-\theta^T x}} \in [0, 1]$$

USE for classification?

$$h_{\theta}(x) > 0.5 \Rightarrow \text{yes} \quad 1$$

$$\text{o.w.} \Rightarrow \text{no} \quad 0$$

LINEAR regression (GAUSSIAN fixed variance)

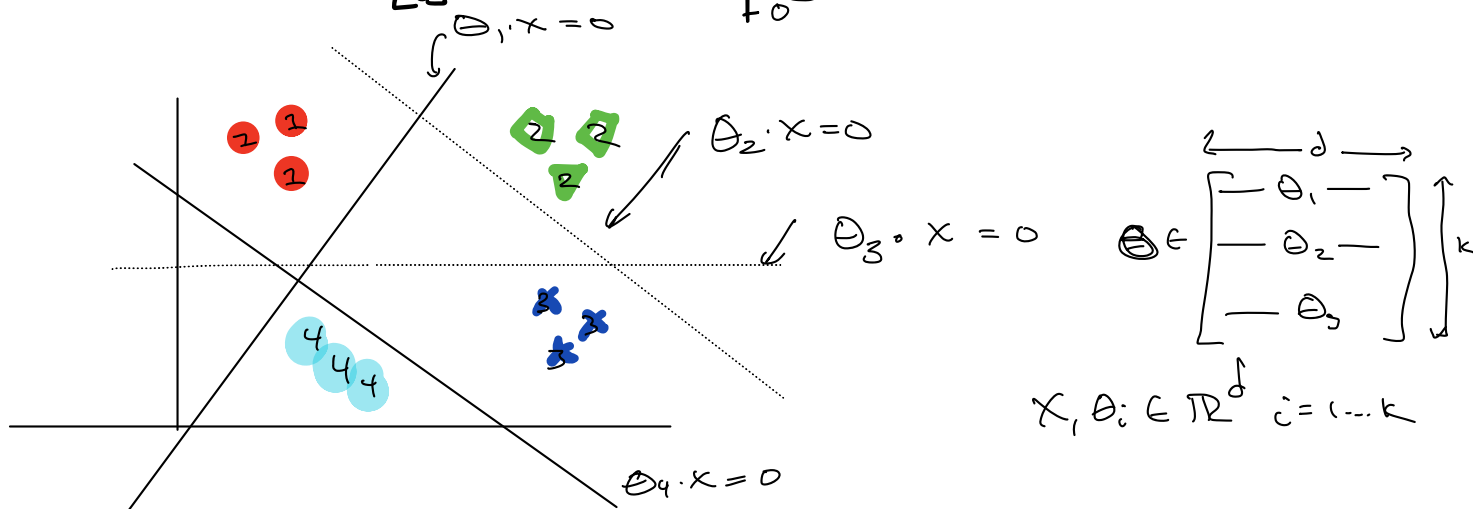
$$h_{\theta}(x) = \mathbb{E}[y | x; \theta] = \mu = \eta = \theta^T x \text{ as before.}$$

Multiclass VIA SOFTMAX (Multinomial)

① DISCRETE VALUES UP TO K $\{car, dog, cat, bus\}$ $K=4$.

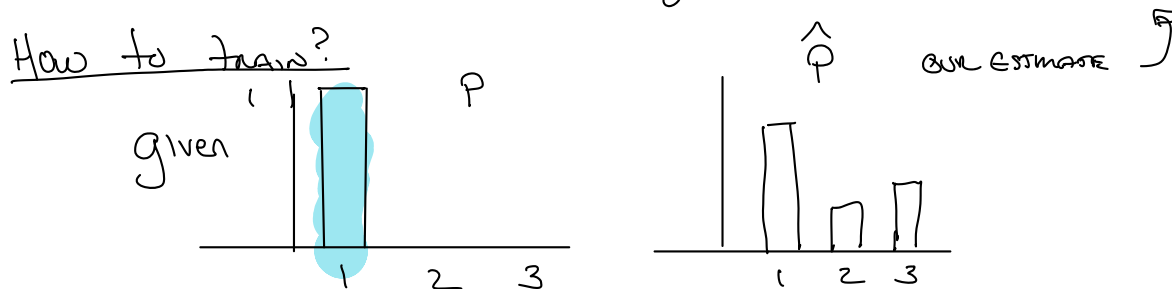
ENCODED AS ONE-HOT vector $\Rightarrow y \in \{0,1\}^K$

E.g. $K=3$ $\begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}$ is class 1 (car) $\begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$ is class 3 (cat)



E.g. $\theta_1 \cdot x = 0.7$ Convert to prob $e^{0.7} \approx 2.013$ normalize 0.57
 $\theta_2 \cdot x = -0.5 \Rightarrow \exp \Rightarrow e^{-0.5} \approx 0.606 \Rightarrow 0.17$
 $\theta_3 \cdot x = -0.1 \Rightarrow e^{-0.1} \approx 0.904 \Rightarrow 0.256$

$$P(y=x | x; \theta) = \frac{\exp(\theta_k \cdot x)}{\sum_{j=1}^K \exp(\theta_j \cdot x)}$$



"the label is 1"

$$\min \text{Cross Entropy}(p, \hat{p}) = - \sum_{y=1}^K p(y) \log(\hat{p}(y_i))$$

ground truth is i $= -\log(\hat{p}(y_i))$ ground truth

$$\text{Just do gradient descent} = -\log \frac{\exp(\theta_i \cdot x)}{\sum_{j=1} \exp(\theta_j \cdot x)}$$



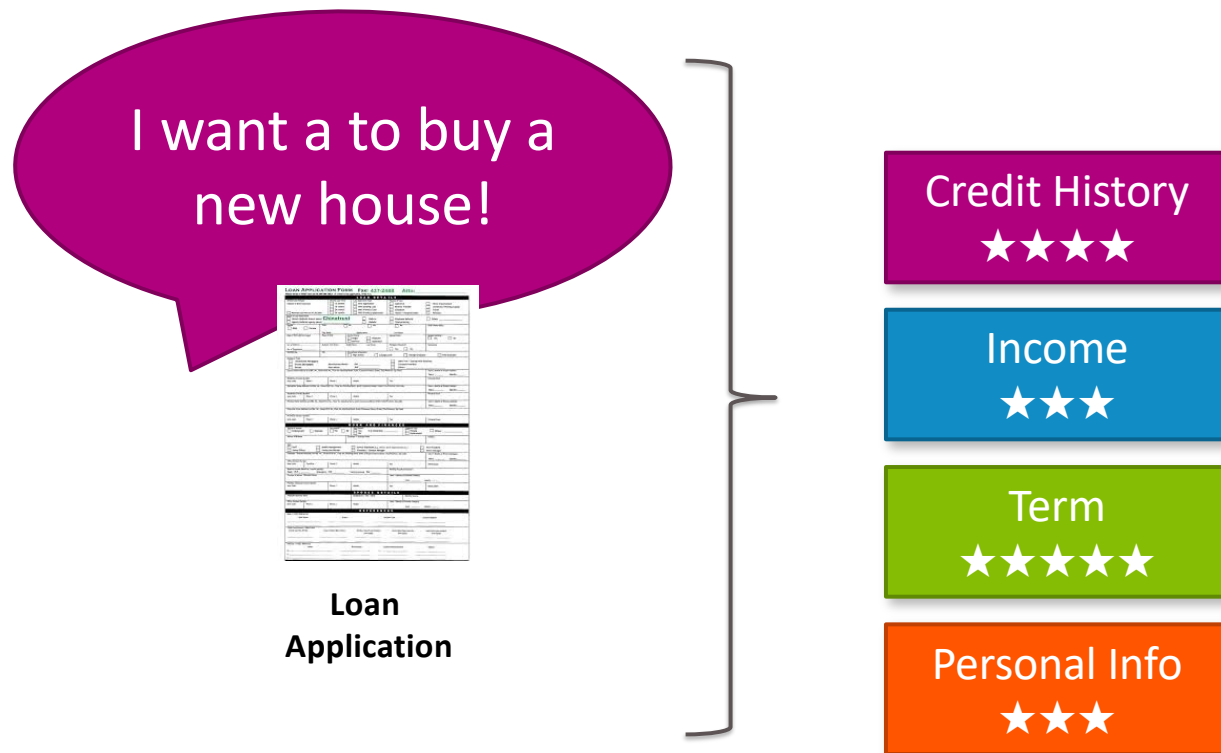
Decision Trees

CS229: Machine Learning
Carlos Guestrin
Stanford University

Slides include content developed by and co-developed with Emily Fox

Predicting potential loan defaults

What makes a loan risky?

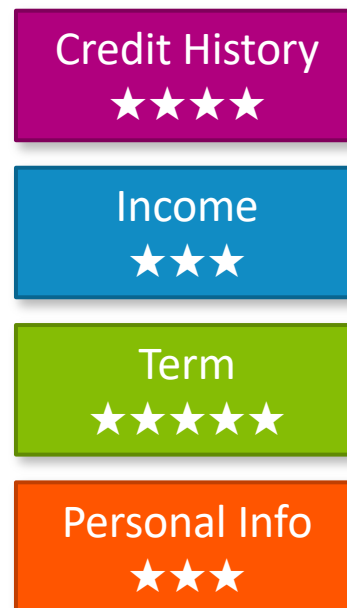


Credit history explained

Did I pay previous
loans on time?



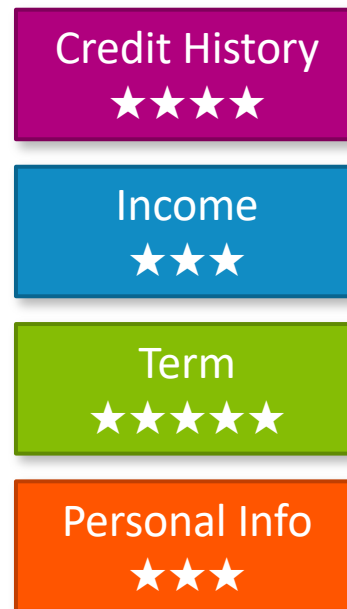
Example: excellent,
good, or fair



Income

What's my income?

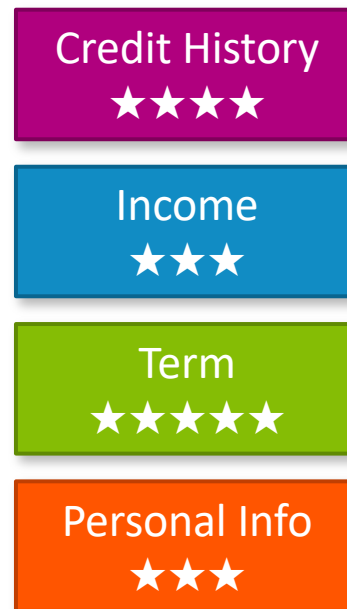
Example:
\$80K per year



Loan terms

How soon do I need to pay the loan?

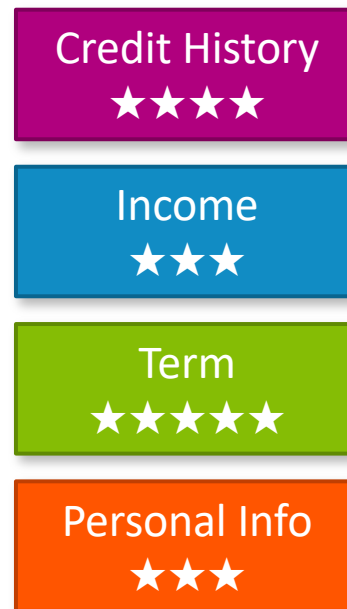
Example: 3 years,
5 years,...



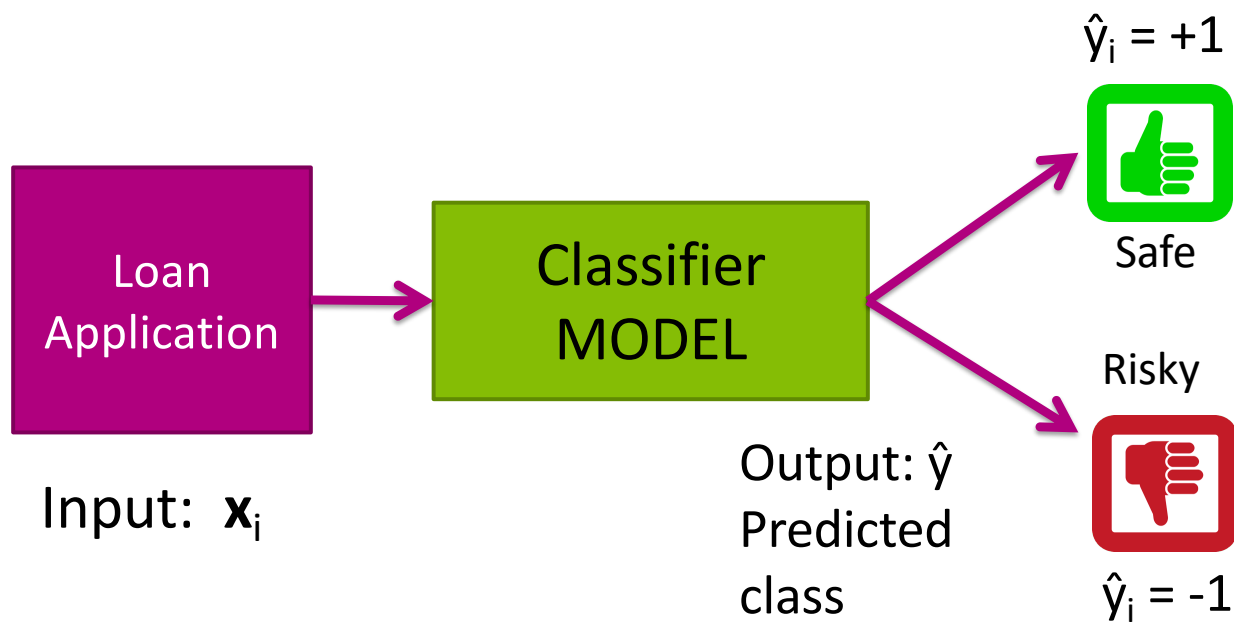
Personal information

Age, reason for the loan,
marital status,...

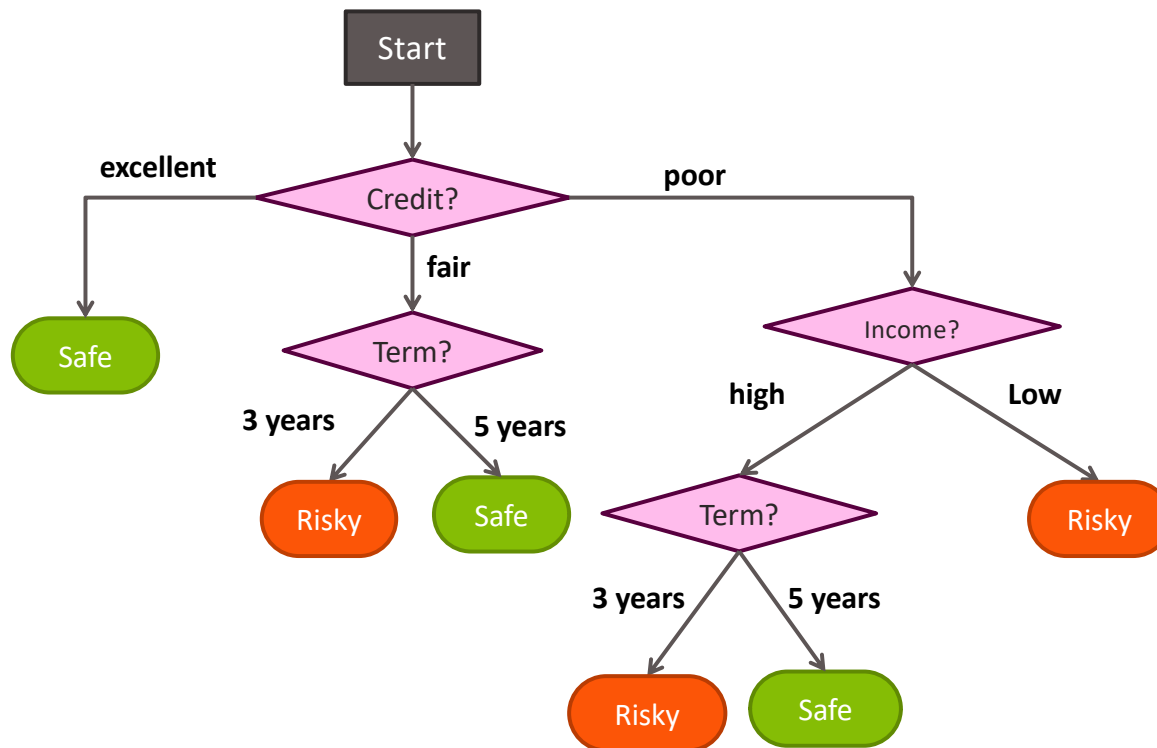
Example: Home loan for a
married couple



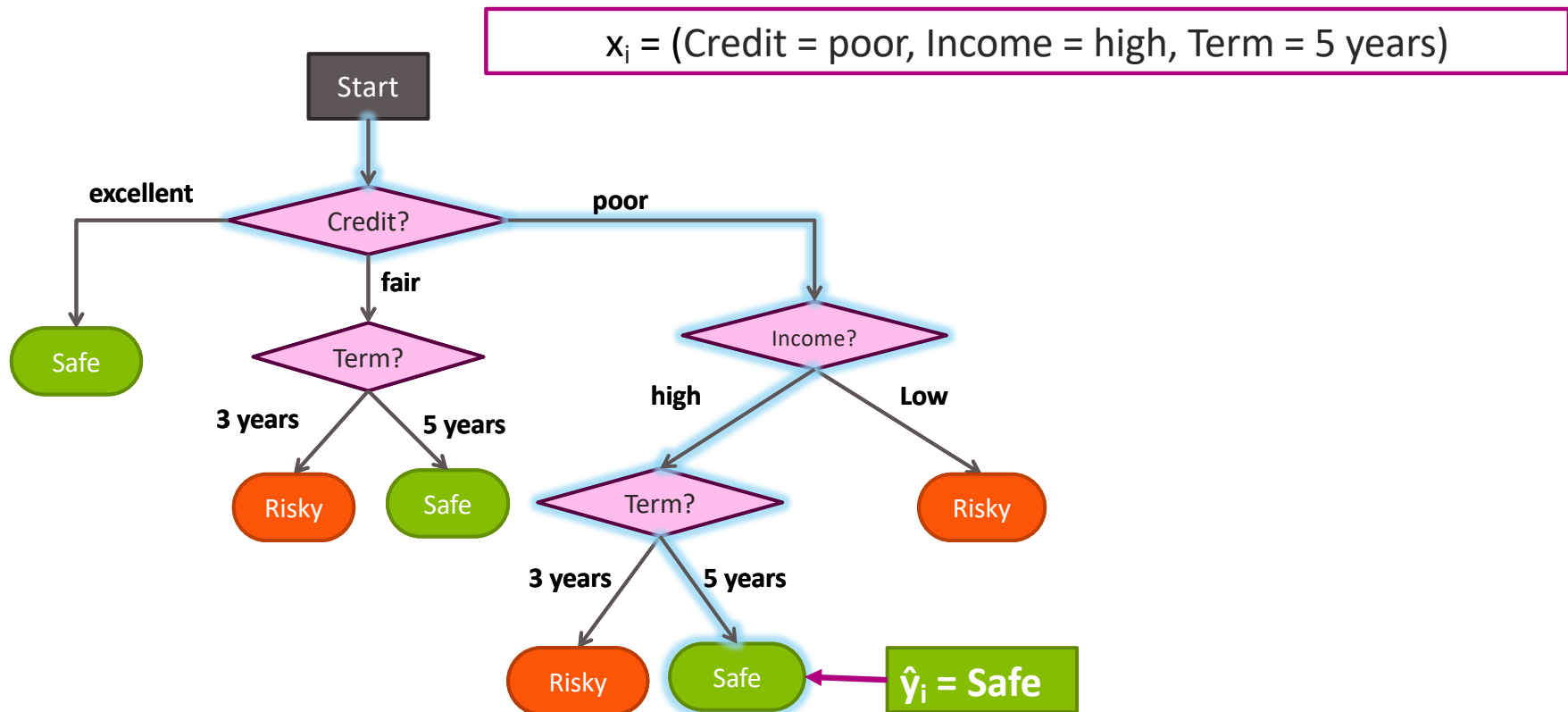
Classifier review



This module ... decision trees



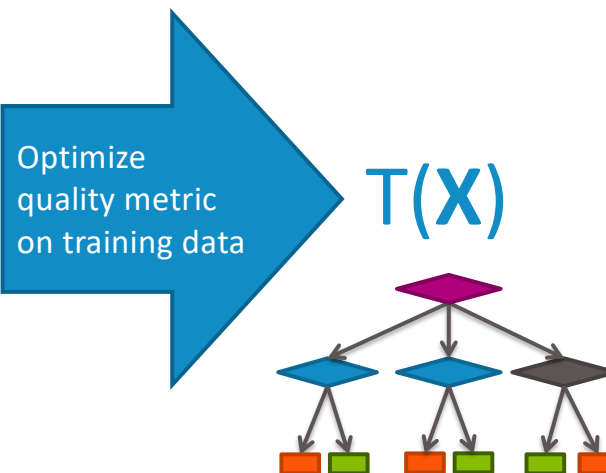
Scoring a loan application



Decision tree learning problem

Training data: N observations (x_i, y_i)

Credit	Term	Income	y
excellent	3 yrs	high	safe
fair	5 yrs	low	risky
fair	3 yrs	high	safe
poor	5 yrs	high	risky
excellent	3 yrs	low	risky
fair	5 yrs	low	safe
poor	3 yrs	high	risky
poor	5 yrs	low	safe
fair	3 yrs	high	safe



Quality metric: Classification error

- Error measures fraction of mistakes

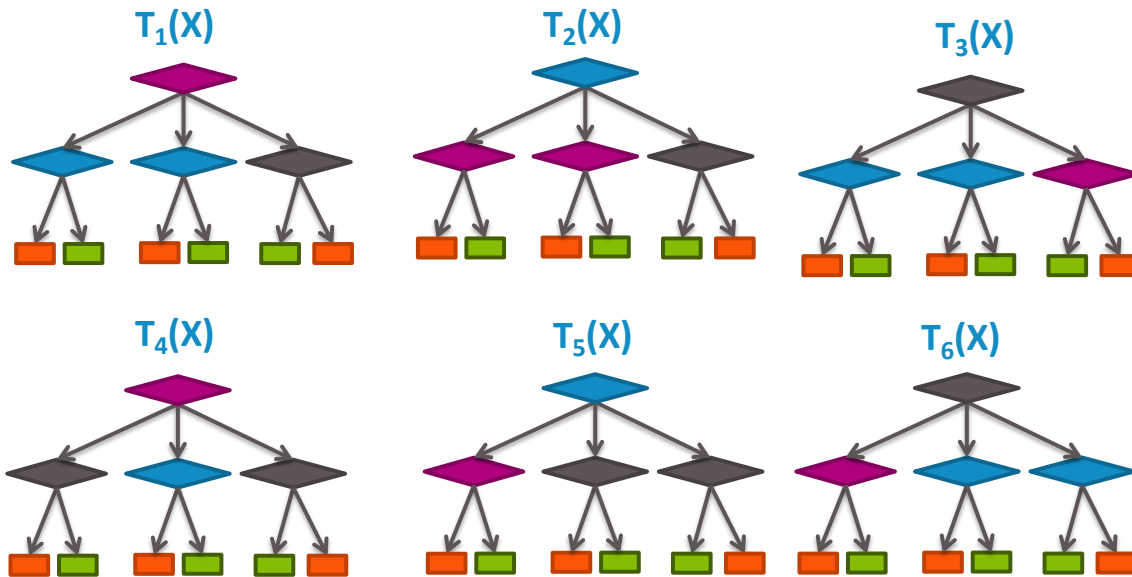
$$\text{Error} = \frac{\text{\# incorrect predictions}}{\text{\# examples}}$$

- Best possible value : 0.0
- Worst possible value: 1.0

How do we find the best tree?

Exponentially large number of possible trees
makes decision tree learning **hard**!

Learning the smallest
decision tree is an
NP-hard problem
[Hyafil & Rivest '76]



Greedy decision tree learning

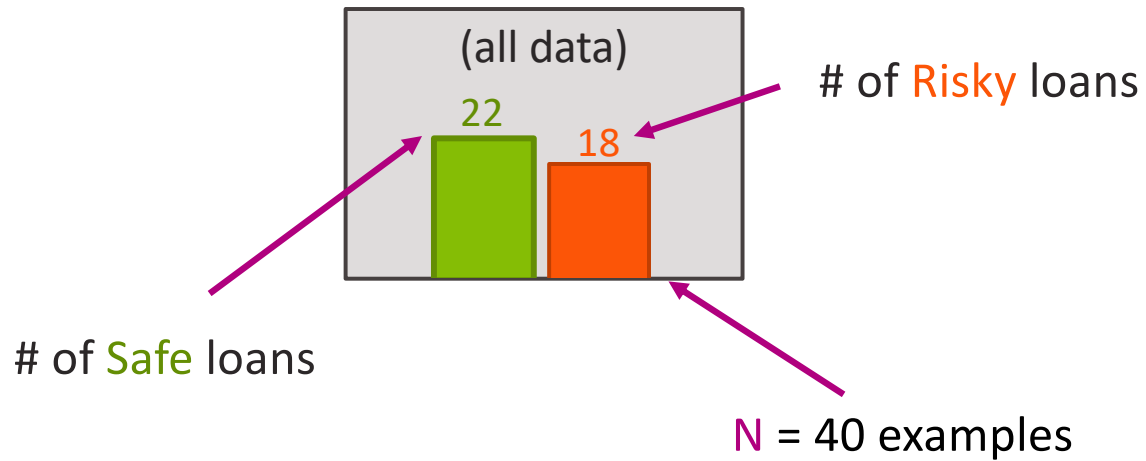
Our training data table

Assume $N = 40$, 3 features

Credit	Term	Income	y
excellent	3 yrs	high	safe
fair	5 yrs	low	risky
fair	3 yrs	high	safe
poor	5 yrs	high	risky
excellent	3 yrs	low	risky
fair	5 yrs	low	safe
poor	3 yrs	high	risky
poor	5 yrs	low	safe
fair	3 yrs	high	safe

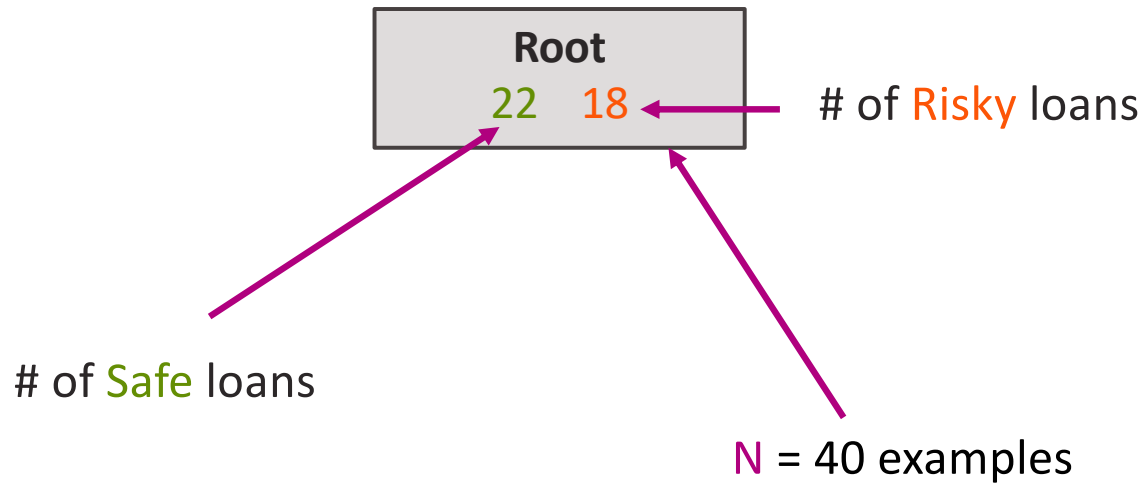
Start with all the data

Loan status: **Safe** **Risky**

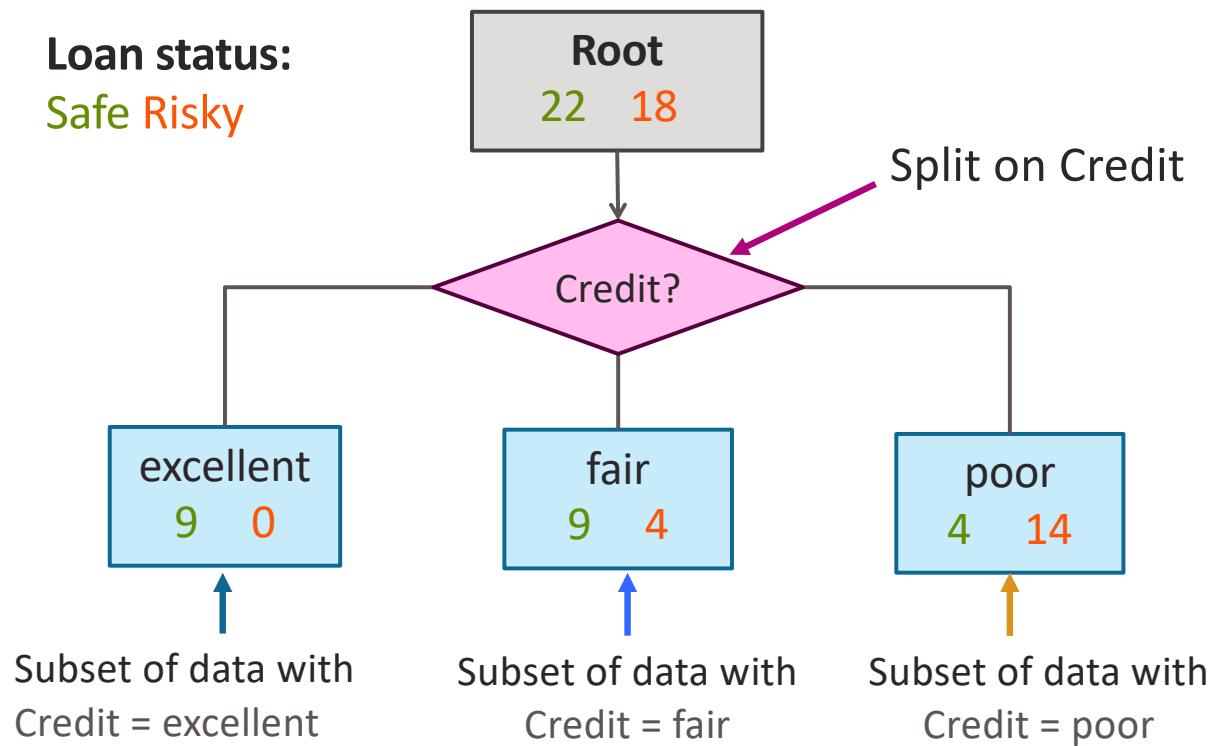


Compact visual notation: Root node

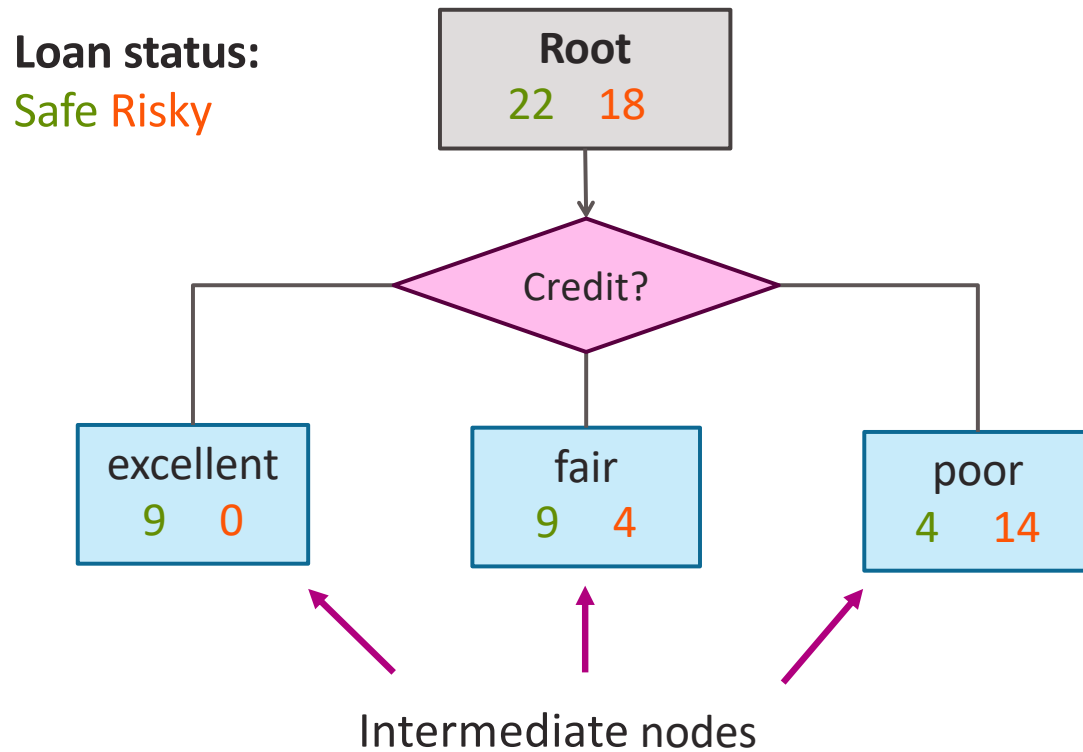
Loan status: **Safe** **Risky**



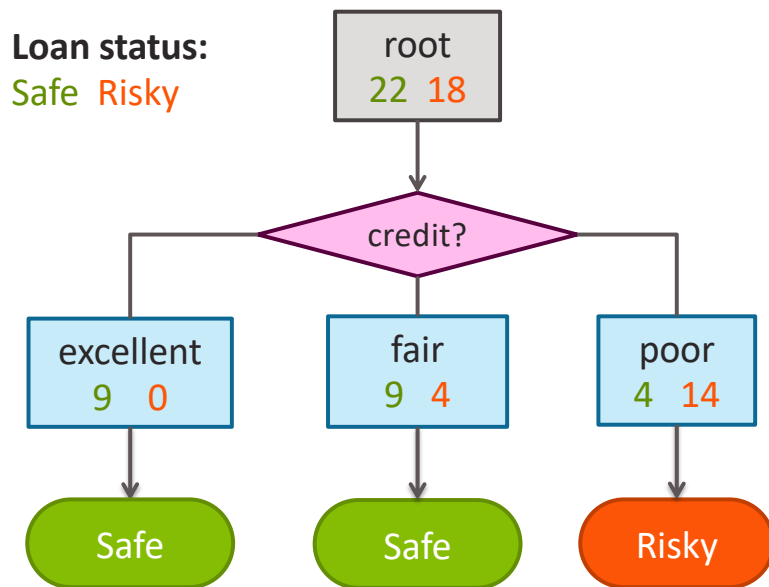
Decision stump: Single level tree



Visual notation: Intermediate nodes



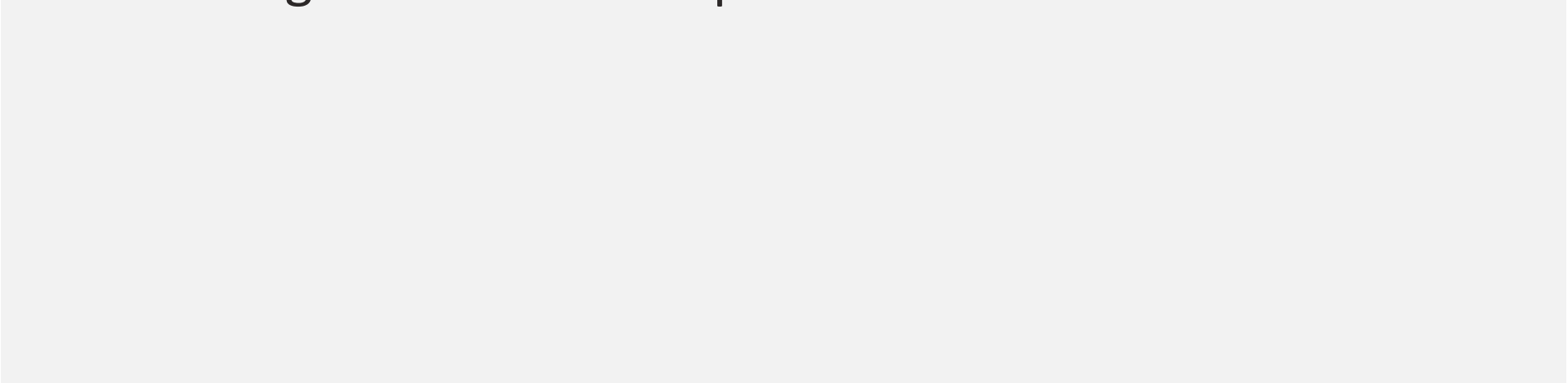
Making predictions with a decision stump



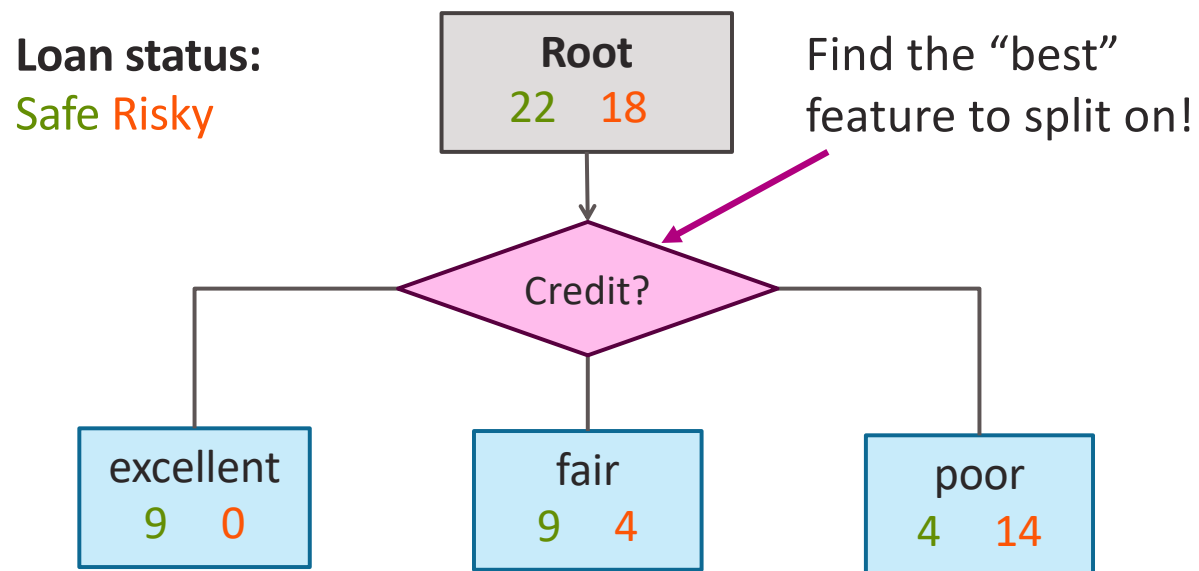
For each intermediate node,
set $\hat{y}$ = majority value



Selecting best feature to split on

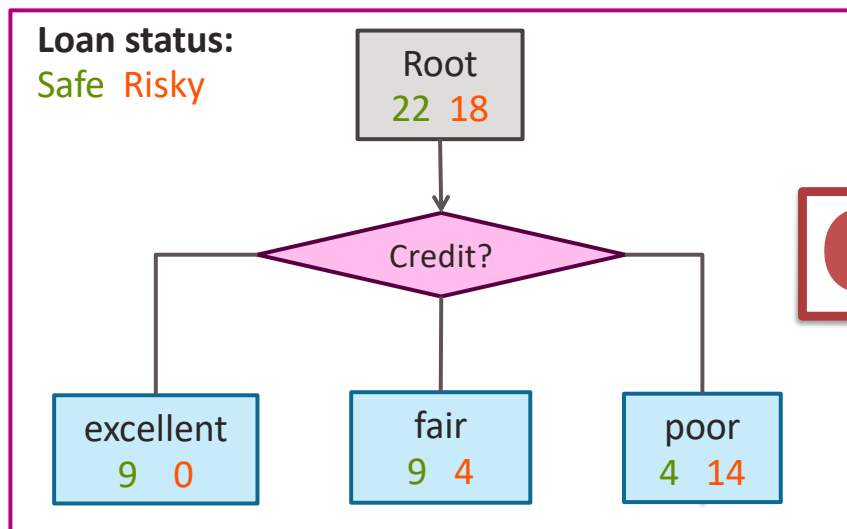


How do we learn a decision stump?



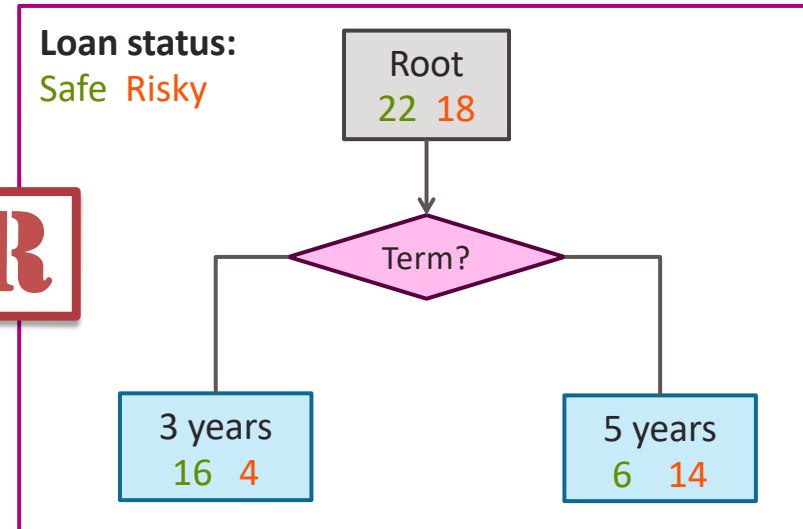
How do we select the best feature?

Choice 1: Split on Credit

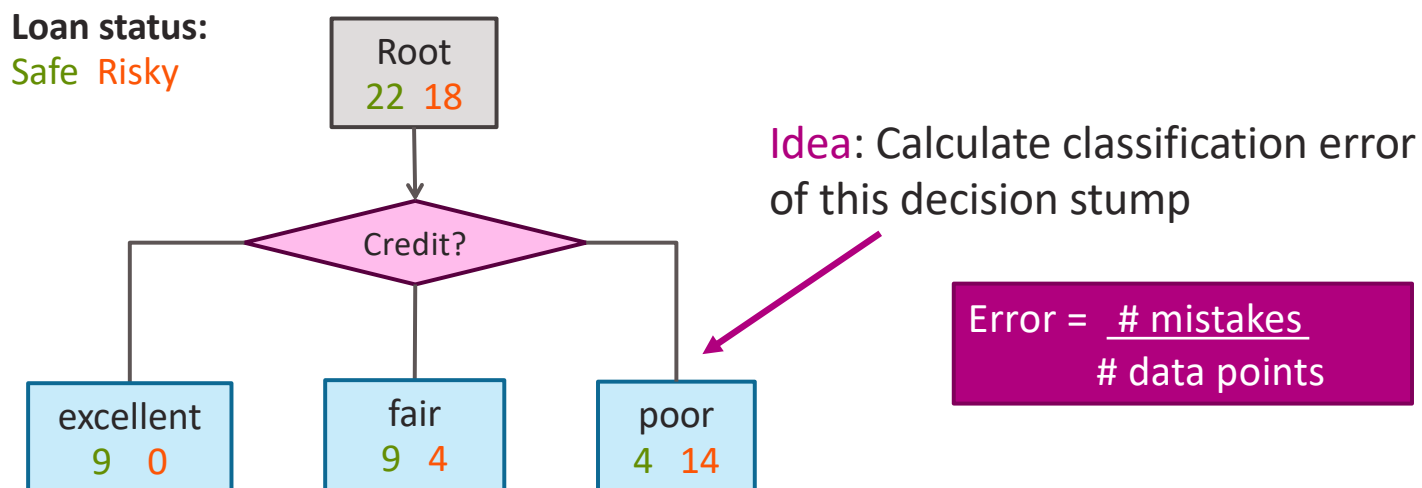


OR

Choice 2: Split on Term

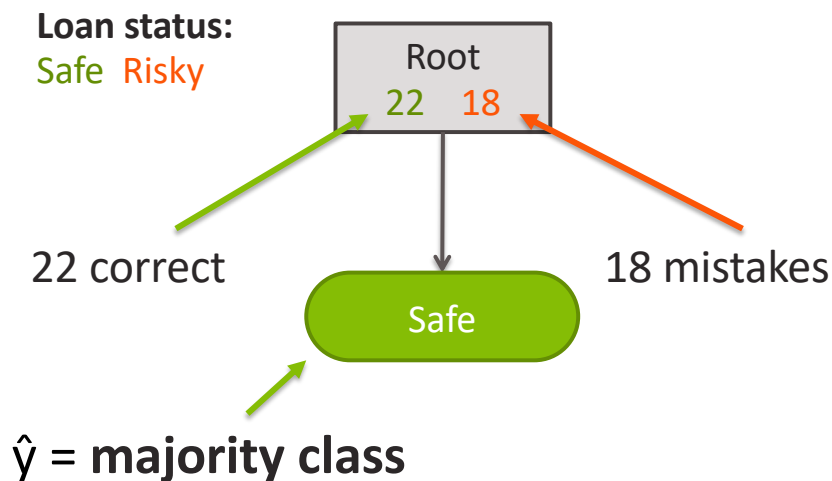


How do we measure effectiveness of a split?



Calculating classification error

- **Step 1:** $\hat{y}$ = class of majority of data in node
- **Step 2:** Calculate classification error of predicting $\hat{y}$ for this data

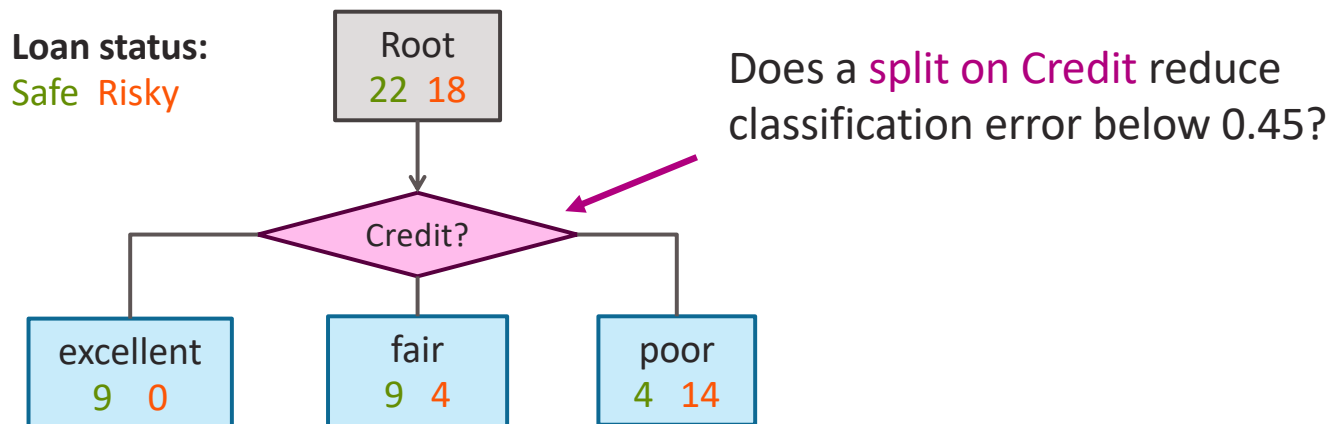


Error = _____
=

Tree	Classification error
(root)	0.45

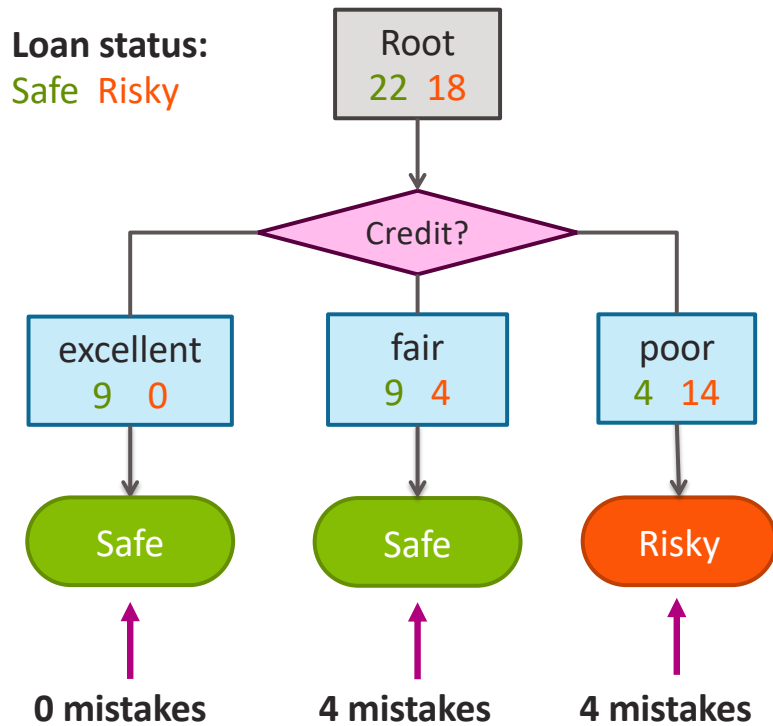
Choice 1: Split on **Credit** history?

Choice 1: Split on Credit



Split on Credit: Classification error

Choice 1: Split on Credit

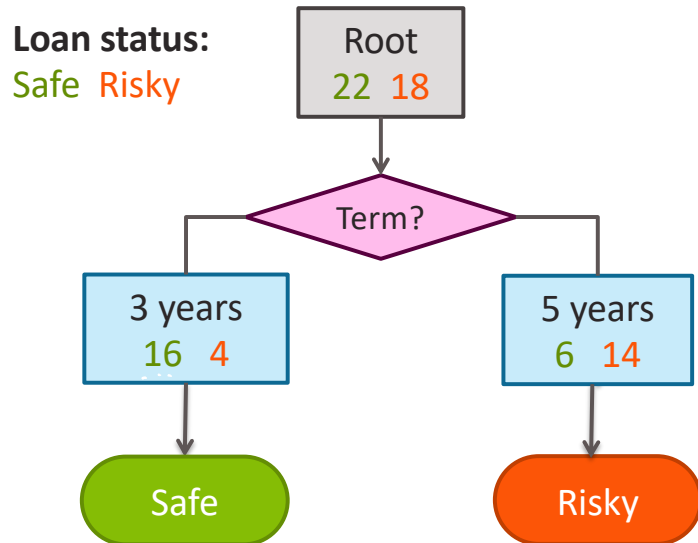


Error = _____
=

Tree	Classification error
(root)	0.45
Split on credit	0.2

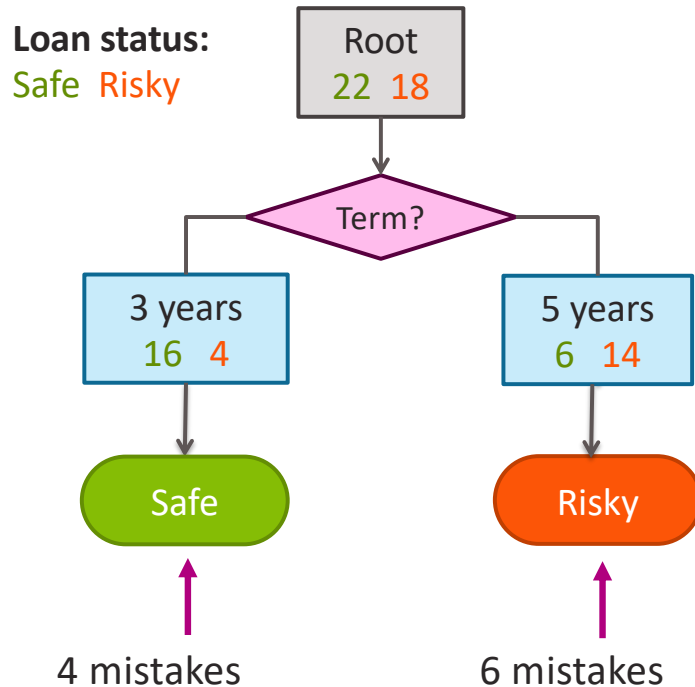
Choice 2: Split on Term?

Choice 2: Split on Term



Evaluating the split on Term

Choice 2: Split on Term



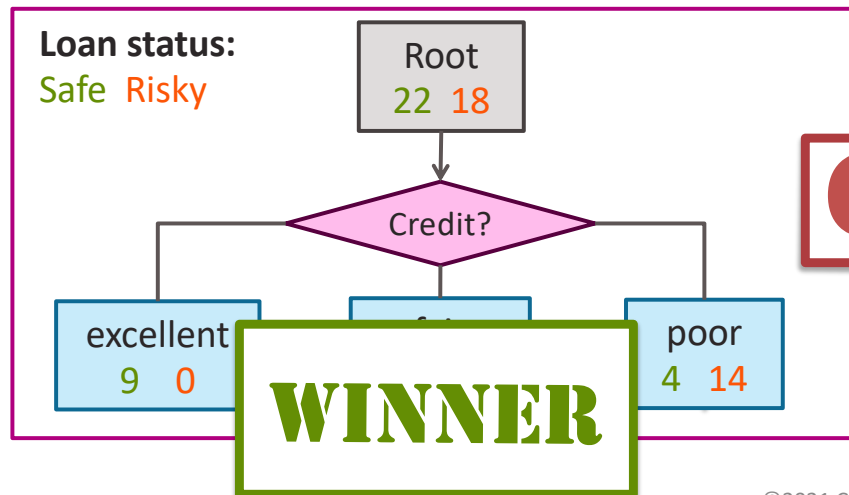
Error = _____
=

Tree	Classification error
(root)	0.45
Split on credit	0.2
Split on term	0.25

Choice 1 vs Choice 2: Comparing split on Credit vs Term

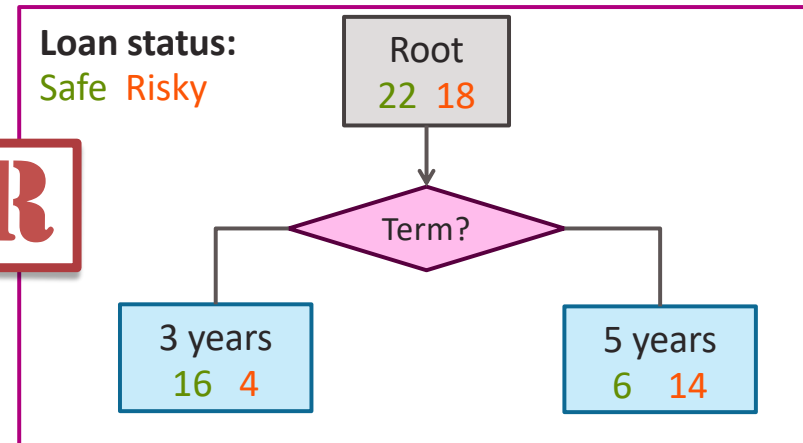
Tree	Classification error
(root)	0.45
split on credit	0.2
split on loan term	0.25

Choice 1: Split on Credit



OR

Choice 2: Split on Term

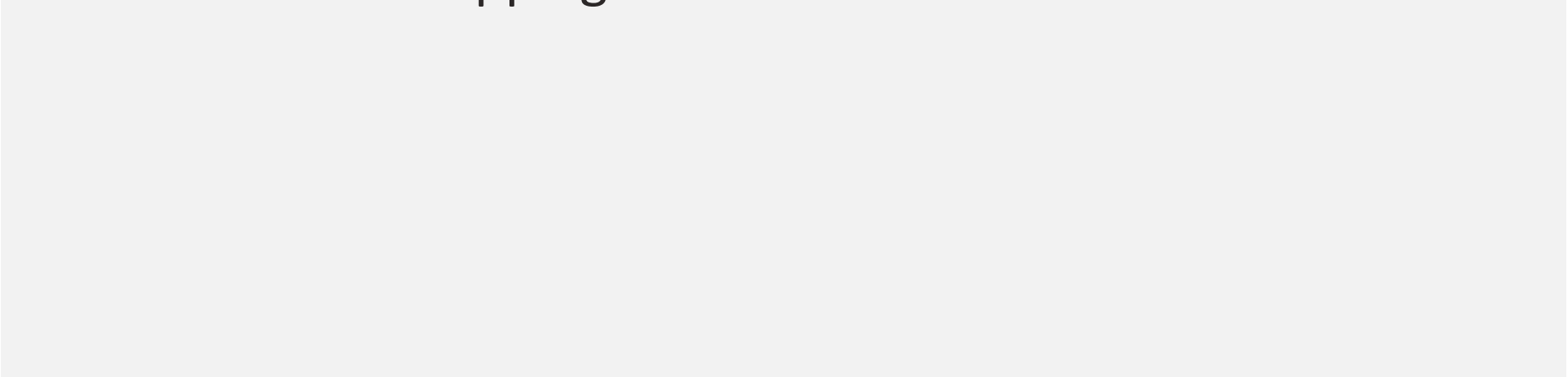


Feature split selection algorithm

- Given a subset of data M (a node in a tree)
- For each feature $h_i(x)$:
 1. Split data of M according to feature $h_i(x)$
 2. Compute classification error of split
- Chose feature $h^*(x)$ with lowest classification error



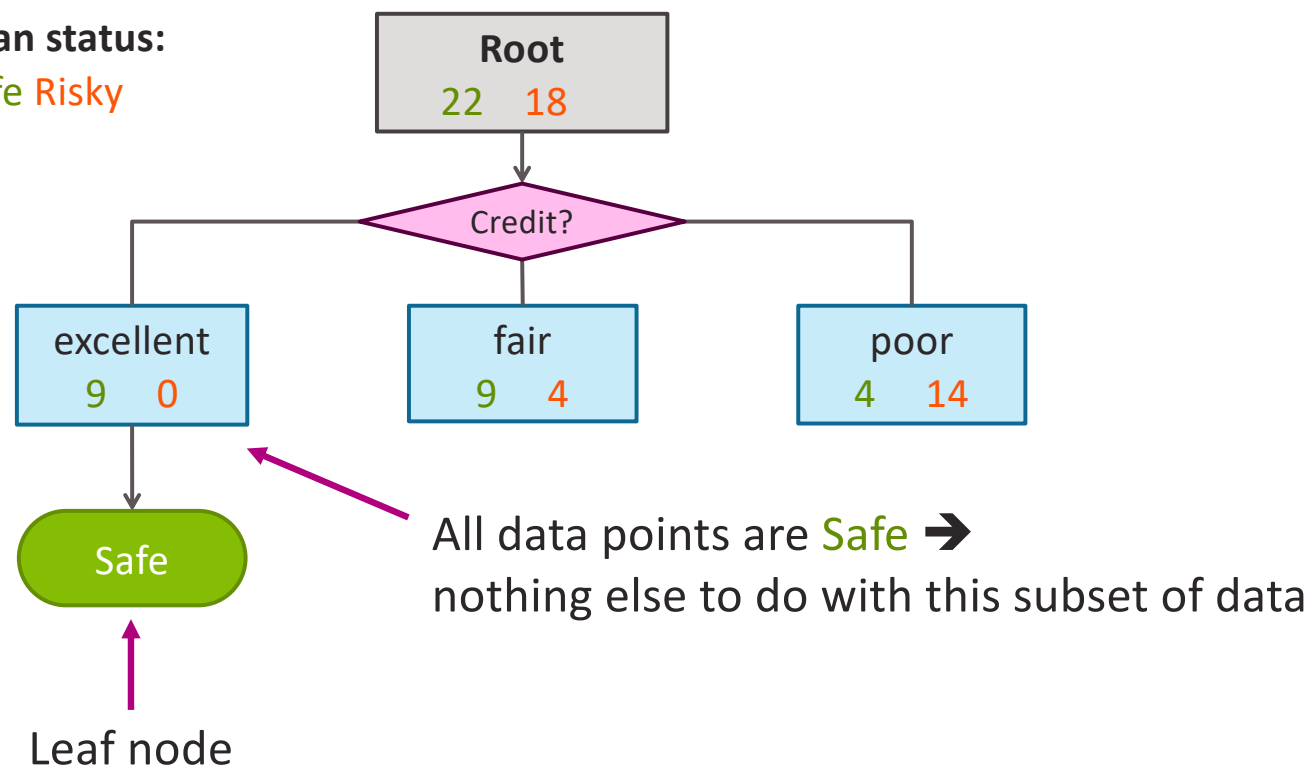
Recursion & Stopping conditions



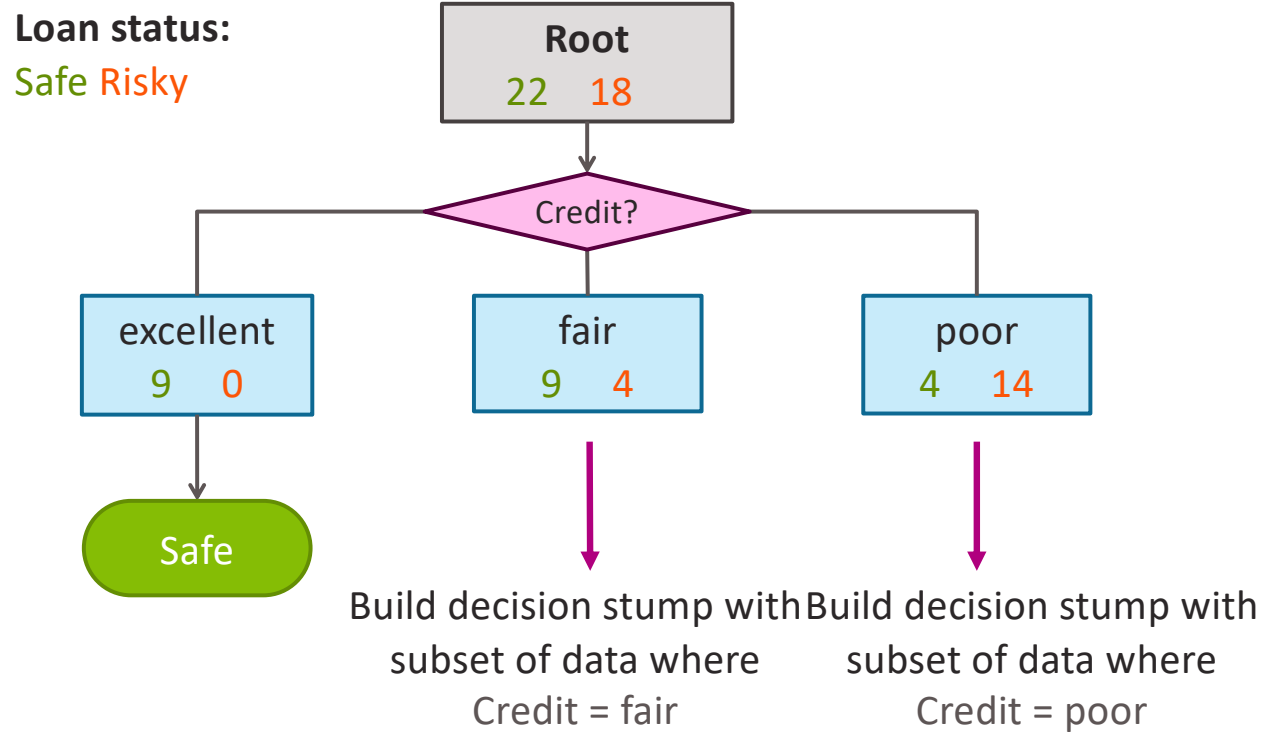
We've learned a decision stump, what next?

Loan status:

Safe Risky

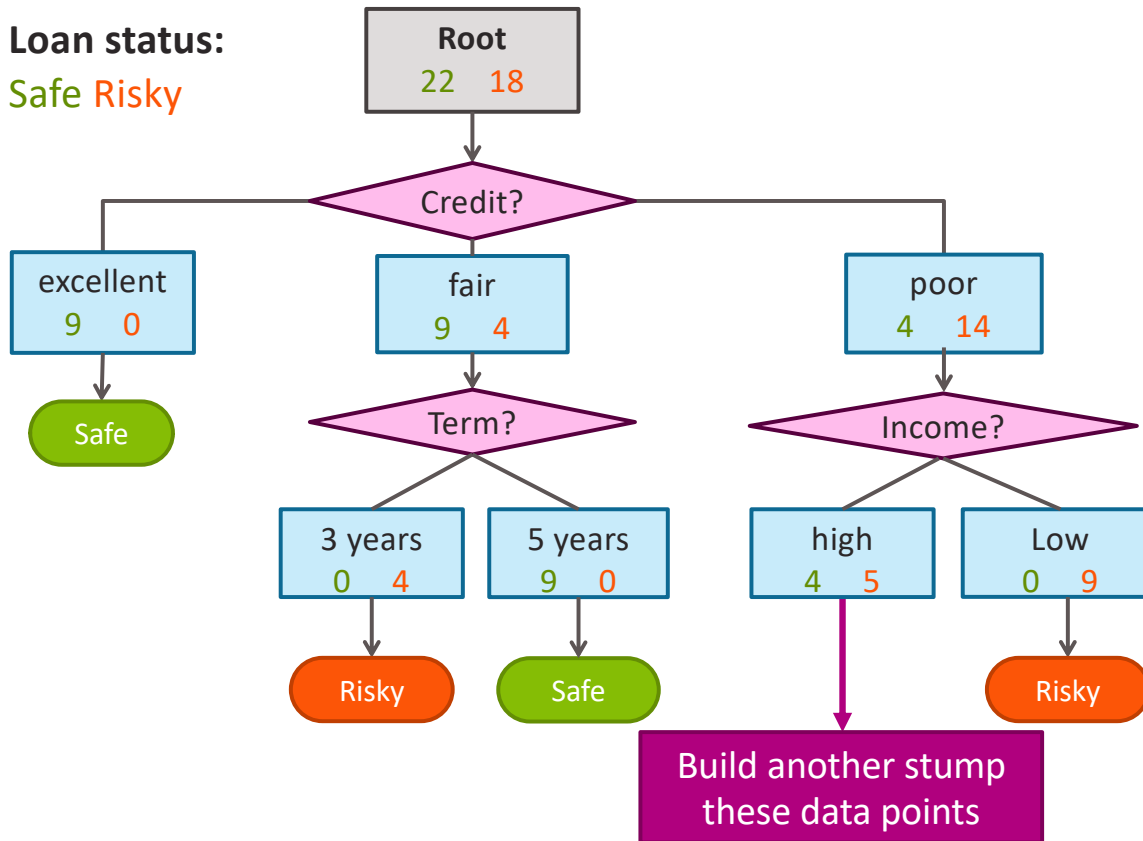


Tree learning = Recursive stump learning

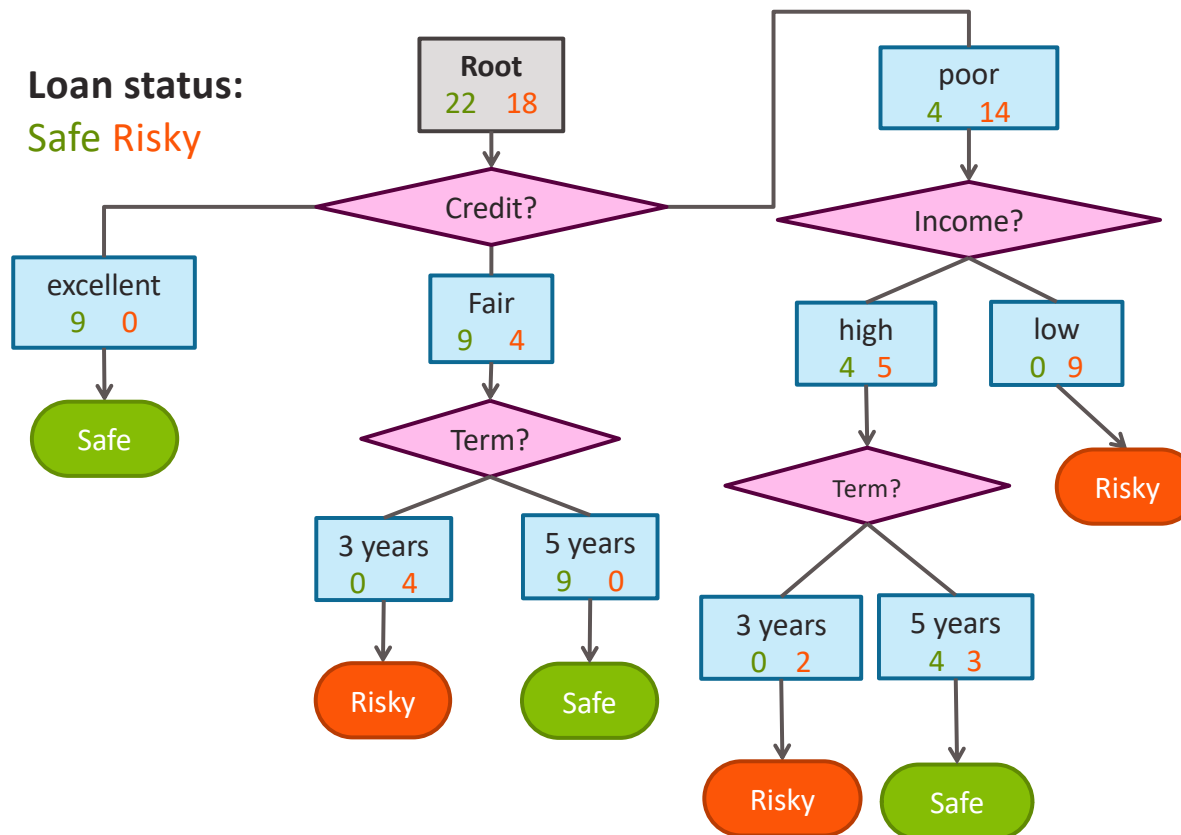


Second level

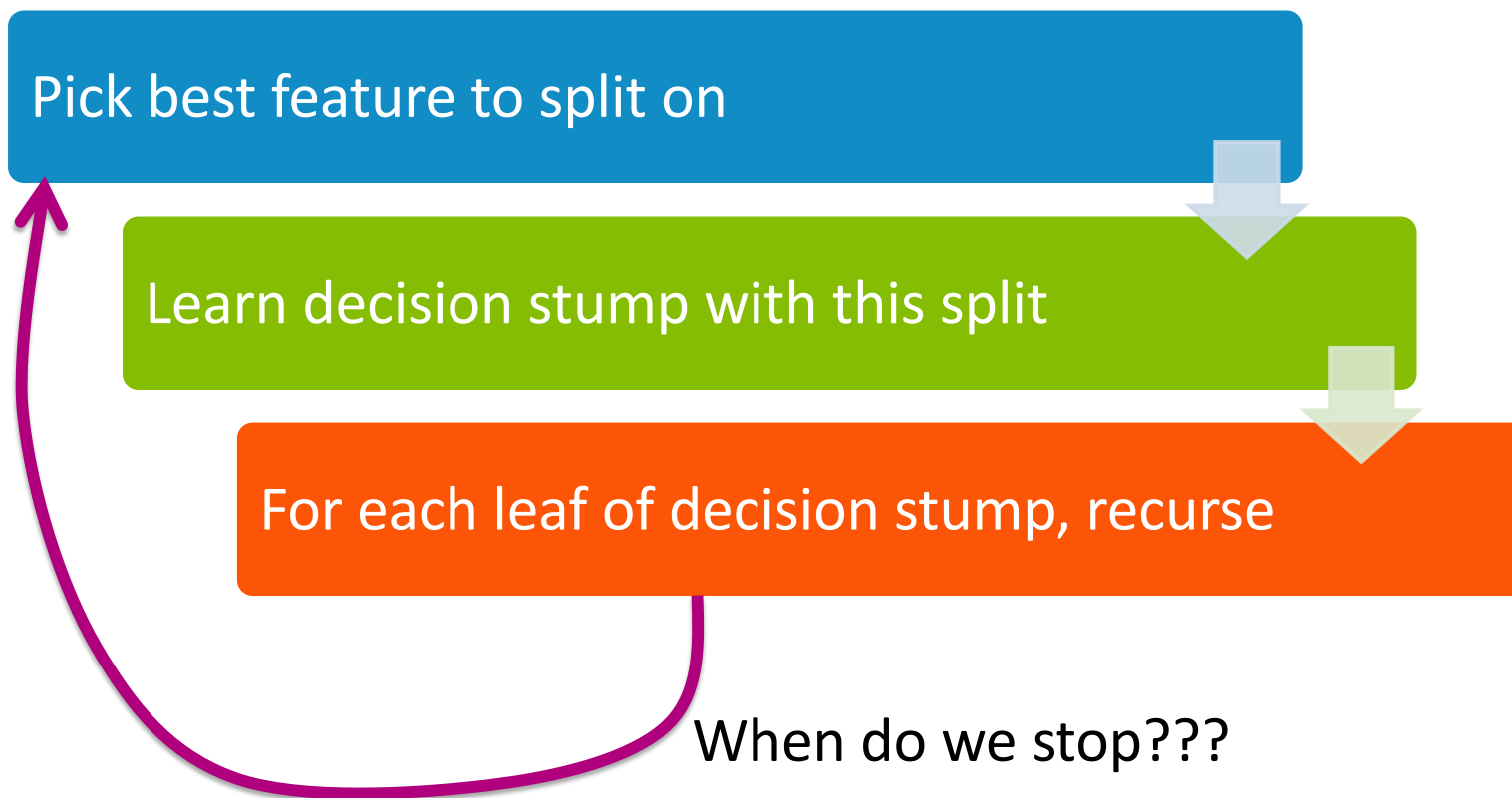
Loan status:
Safe Risky



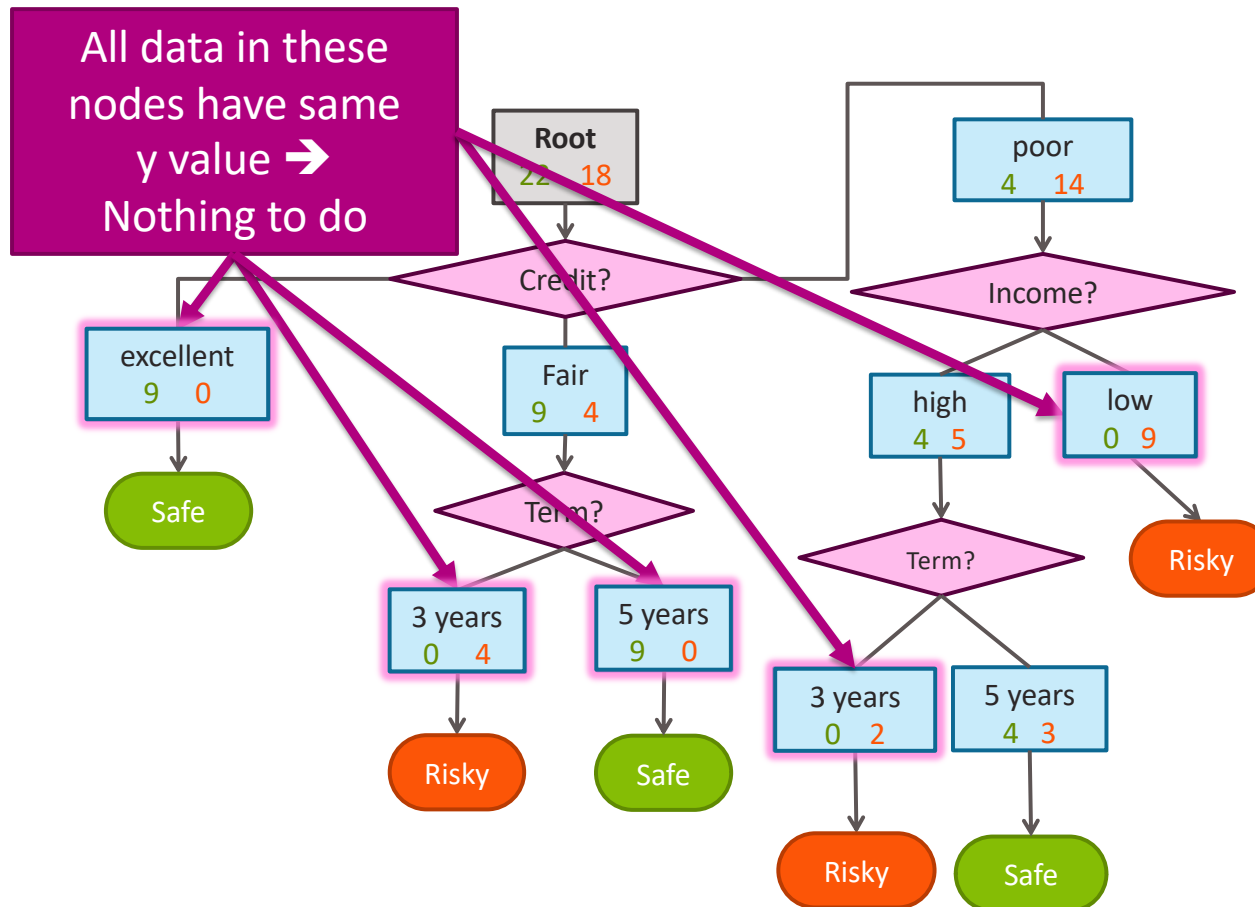
Final decision tree



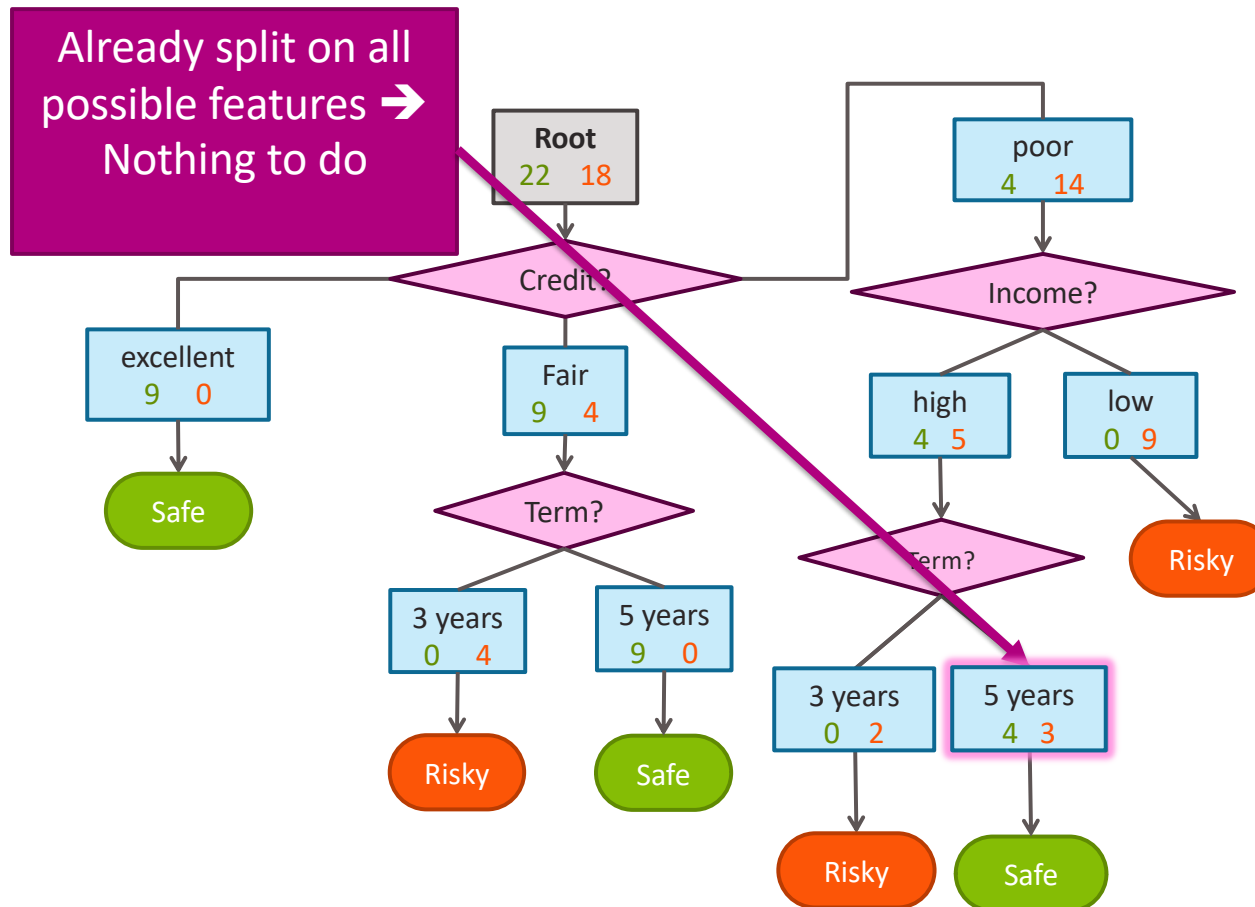
Simple greedy decision tree learning



Stopping condition 1: All data agrees on y



Stopping condition 2: Already split on all features



Greedy decision tree learning

- **Step 1:** Start with an empty tree

- **Step 2:** Select a feature to split data

- For each split of the tree:

- **Step 3:** If nothing more to do, make predictions

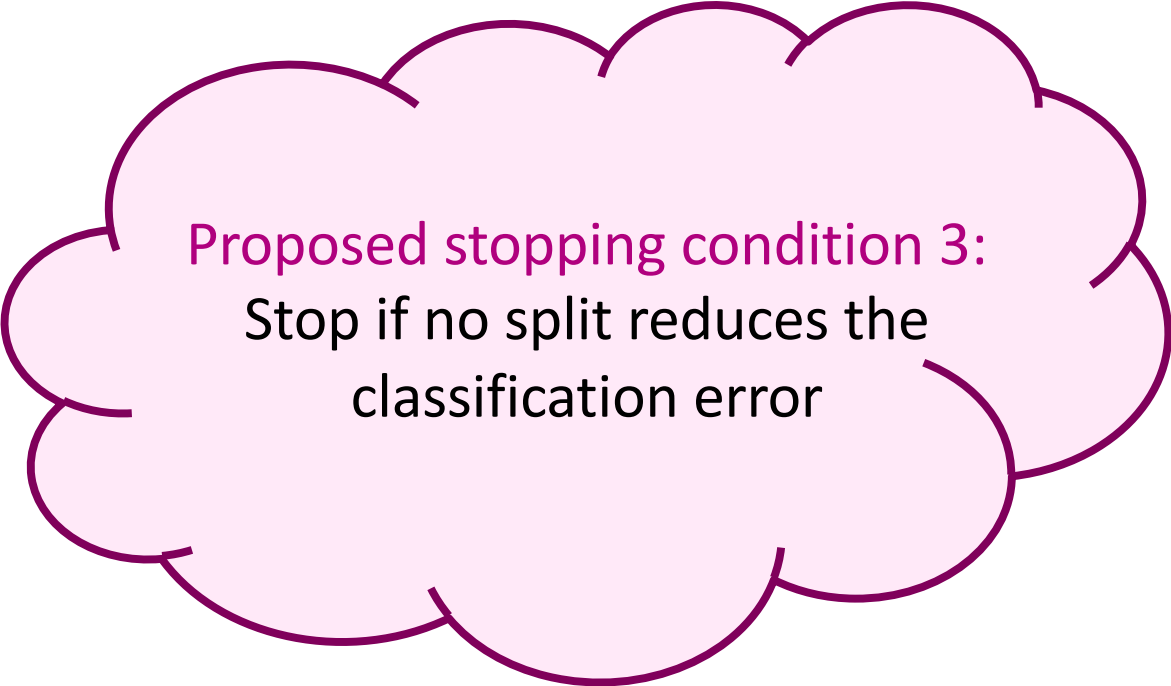
- **Step 4:** Otherwise, go to **Step 2** & continue (recurse) on this split

Pick feature split
leading to lowest
classification error

Stopping conditions
1 & 2

Recursion

Is this a good idea?



Proposed stopping condition 3:
Stop if no split reduces the
classification error

Stopping condition 3: Don't stop if error doesn't decrease???

$$y = x[1] \text{ xor } x[2]$$

x[1]	x[2]	y
False	False	False
False	True	True
True	False	True
True	True	False

y values
True False

Root
2 2

Error = _____
=

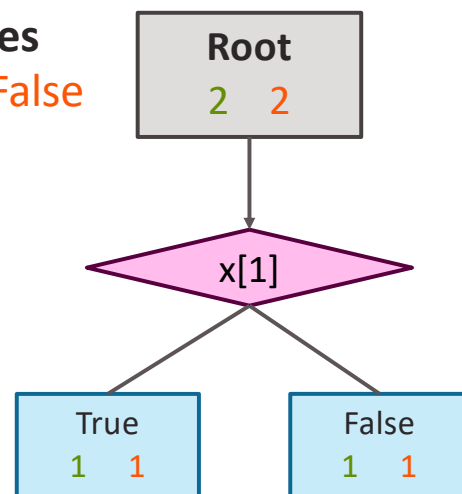
Tree	Classification error
(root)	0.5

Consider split on x[1]

$$y = x[1] \text{ xor } x[2]$$

x[1]	x[2]	y
False	False	False
False	True	True
True	False	True
True	True	False

y values
True False



Error = _____
=

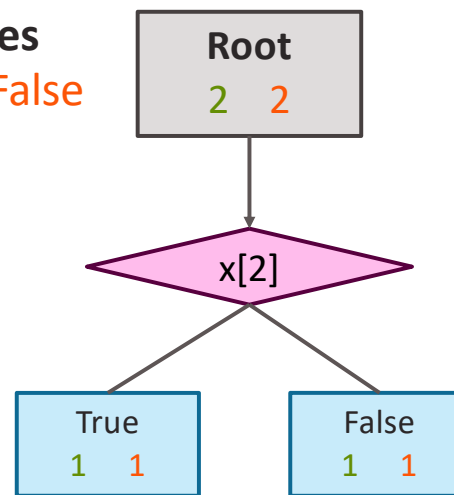
Tree	Classification error
(root)	0.5
Split on x[1]	0.5

Consider split on x[2]

$$y = x[1] \text{ xor } x[2]$$

x[1]	x[2]	y
False	False	False
False	True	True
True	False	True
True	True	False

y values
True False



$$\text{Error} = \frac{1+1}{2+2} = 0.5$$

Neither features
improve training error...
Stop now???

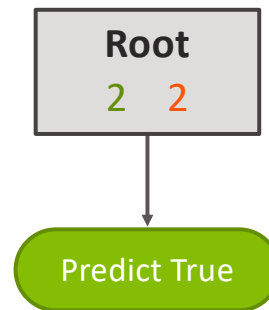
Tree	Classification error
(root)	0.5
Split on x[1]	0.5
Split on x[2]	0.5

Final tree with stopping condition 3

$$y = x[1] \text{ xor } x[2]$$

x[1]	x[2]	y
False	False	False
False	True	True
True	False	True
True	True	False

y values
True False



Tree	Classification error
with stopping condition 3	0.5

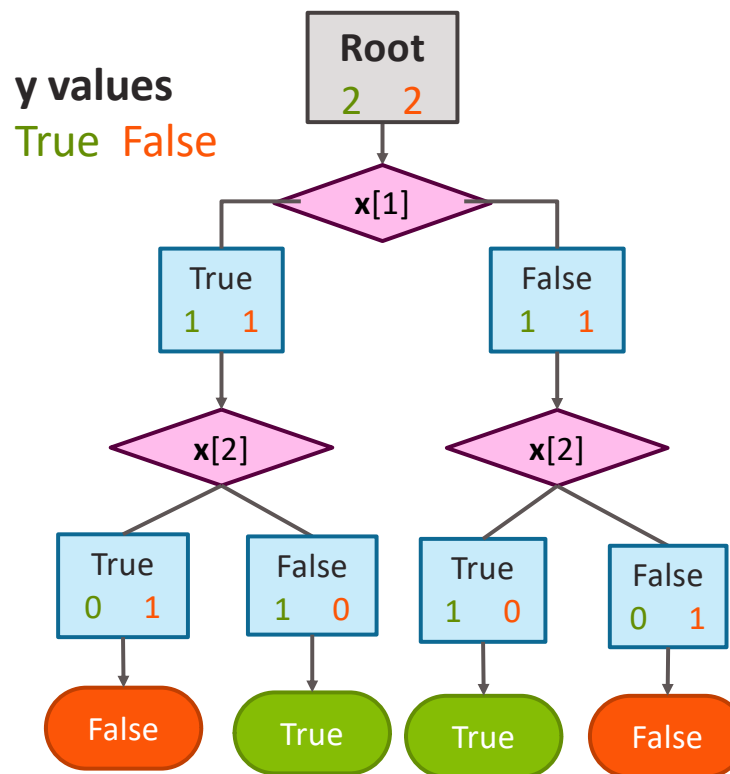
Without stopping condition 3

Condition 3 (stopping when training error doesn't improve) is not recommended!

$$y = x[1] \text{ xor } x[2]$$

x[1]	x[2]	y
False	False	False
False	True	True
True	False	True
True	True	False

Tree	Classification error
with stopping condition 3	0.5
without stopping condition 3	



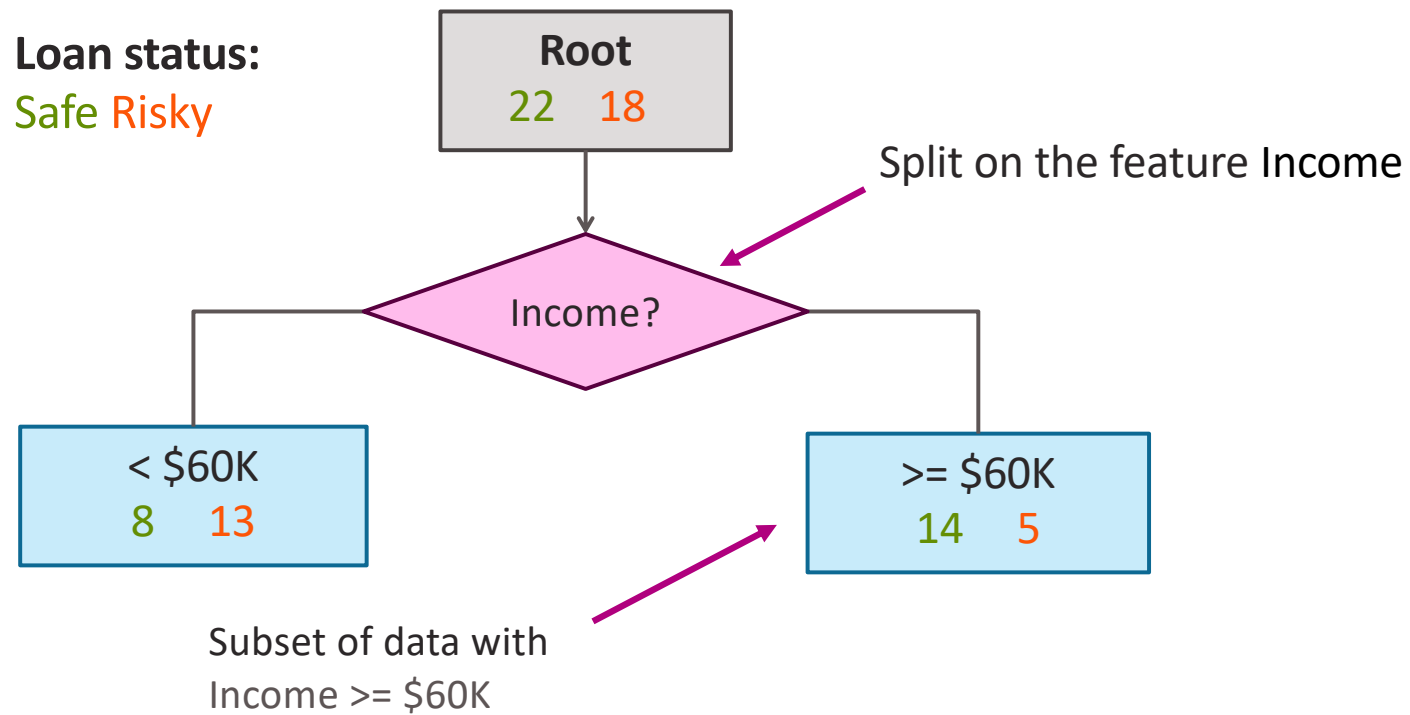
Decision tree learning:

Real valued features

How do we use real values inputs?

Income	Credit	Term	y
\$105 K	excellent	3 yrs	Safe
\$112 K	good	5 yrs	Risky
\$73 K	fair	3 yrs	Safe
\$69 K	excellent	5 yrs	Safe
\$217 K	excellent	3 yrs	Risky
\$120 K	good	5 yrs	Safe
\$64 K	fair	3 yrs	Risky
\$340 K	excellent	5 yrs	Safe
\$60 K	good	3 yrs	Risky

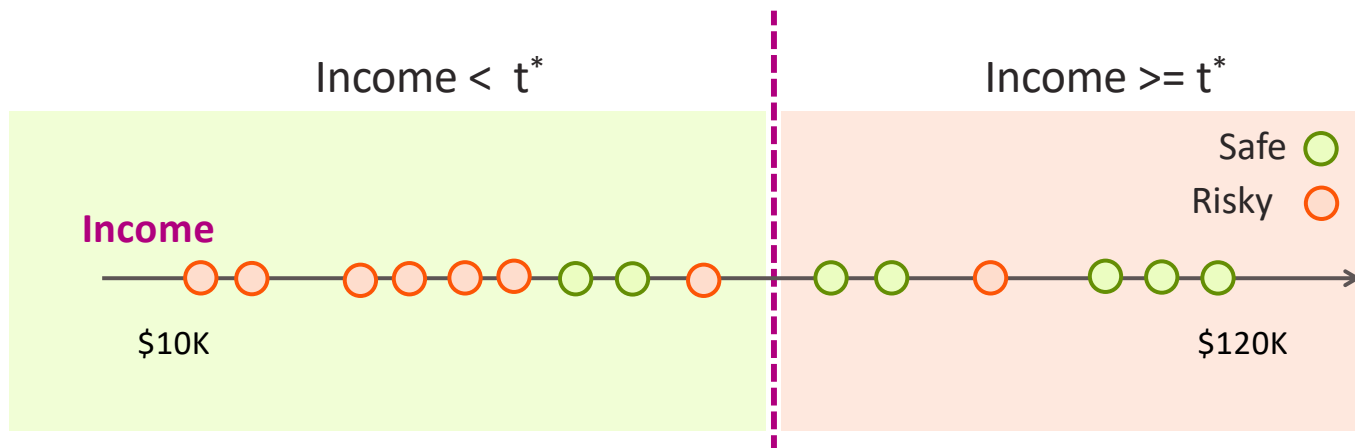
Threshold split



Finding the best threshold split

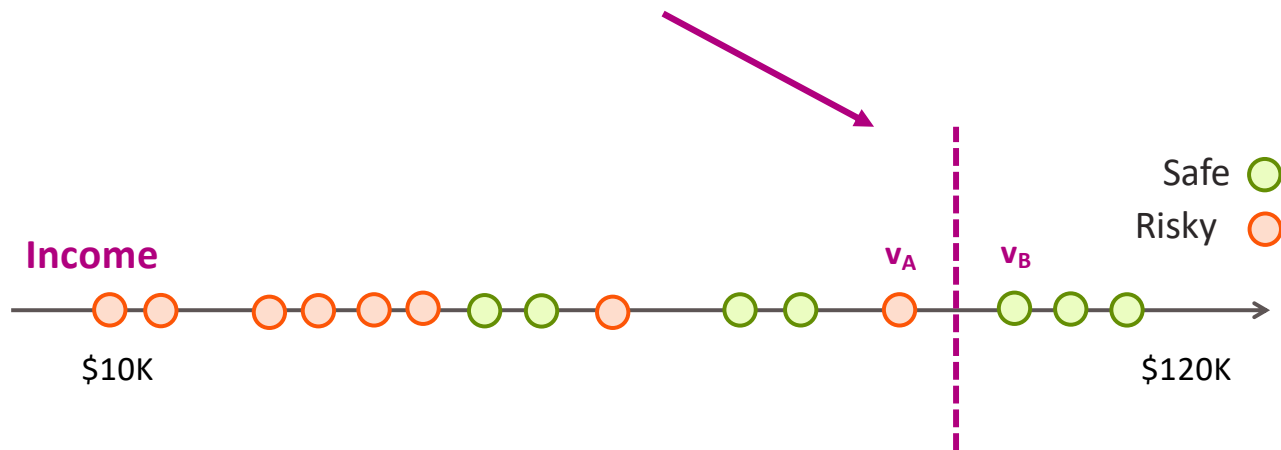
Infinite possible
values of t

Income = t^*



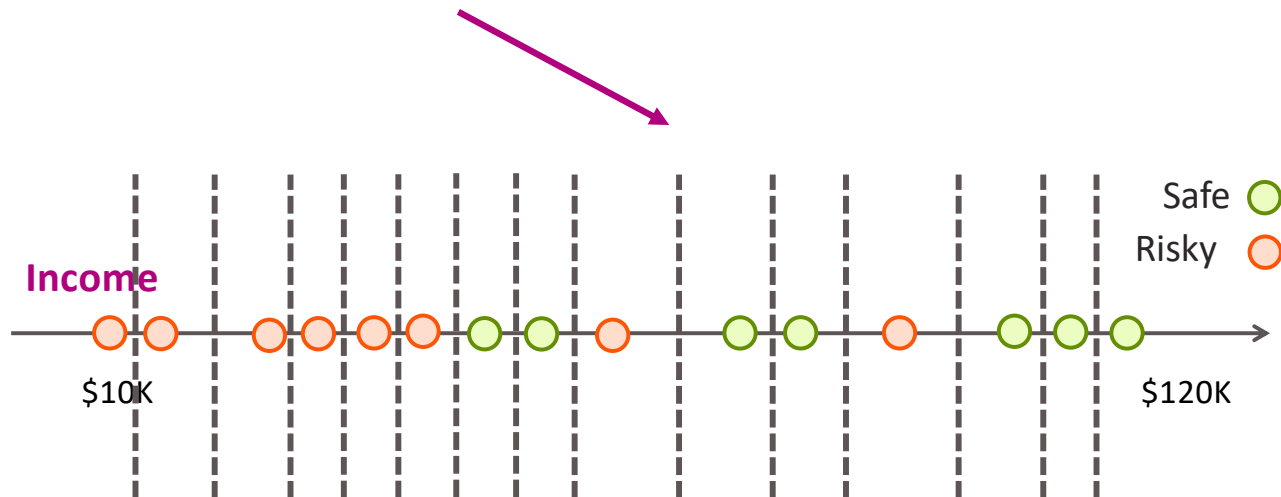
Consider a threshold between points

Same **classification error** for any threshold split between v_A and v_B



Only need to consider mid-points

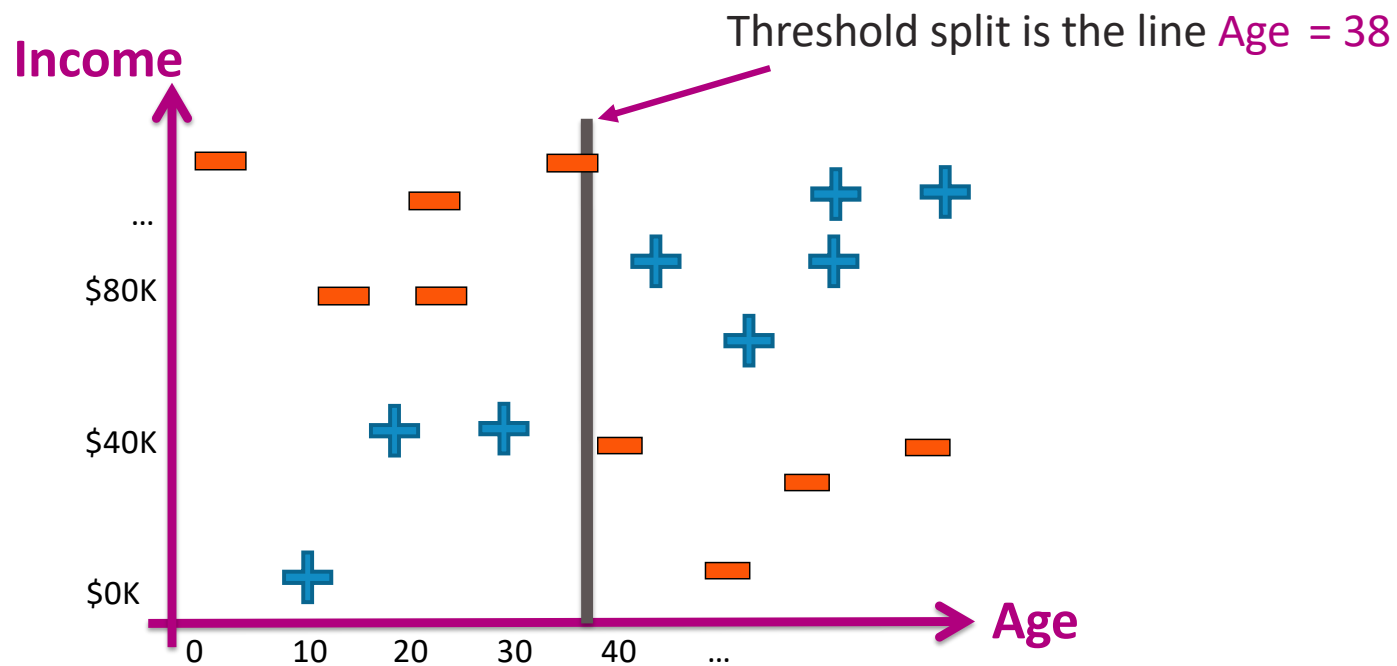
Finite number of splits
to consider



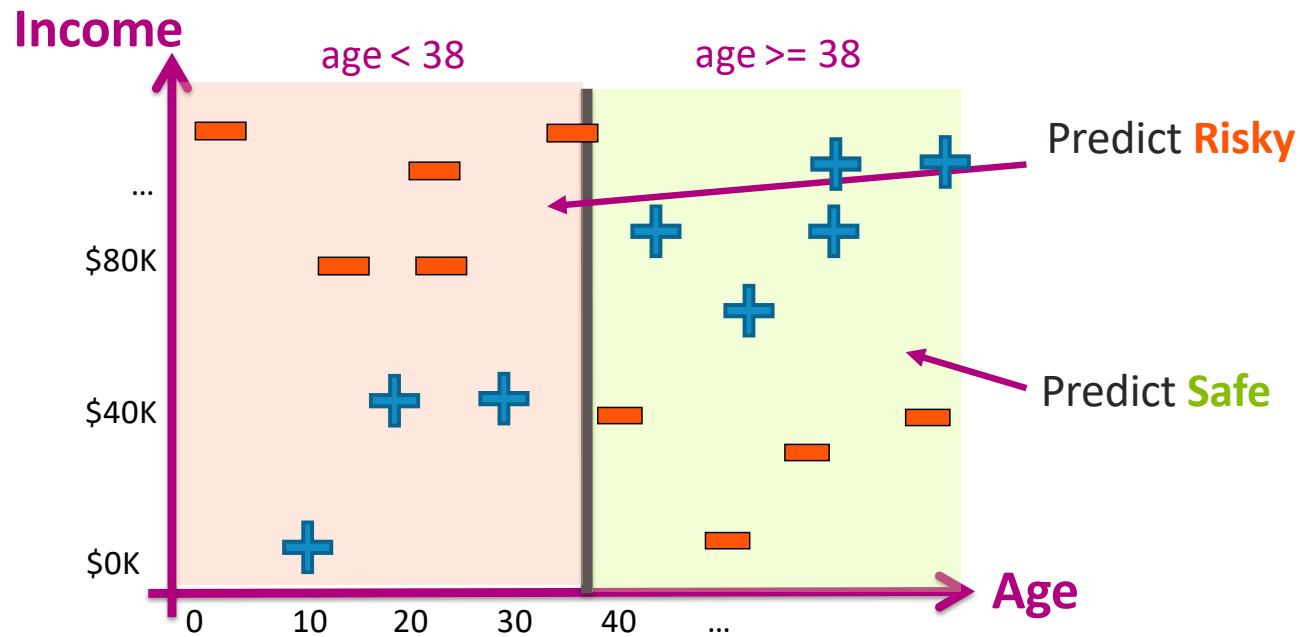
Threshold split selection algorithm

- **Step 1:** Sort the values of a feature $h_j(x)$:
Let $\{\mathbf{v}_1, \mathbf{v}_2, \mathbf{v}_3, \dots \mathbf{v}_N\}$ denote sorted values
- **Step 2:**
 - For $i = 1 \dots N-1$
 - Consider split $t_i = (\mathbf{v}_i + \mathbf{v}_{i+1}) / 2$
 - Compute classification error for threshold split $h_j(x) \geq t_i$
 - Chose the $\mathbf{t}^*$ with the lowest classification error

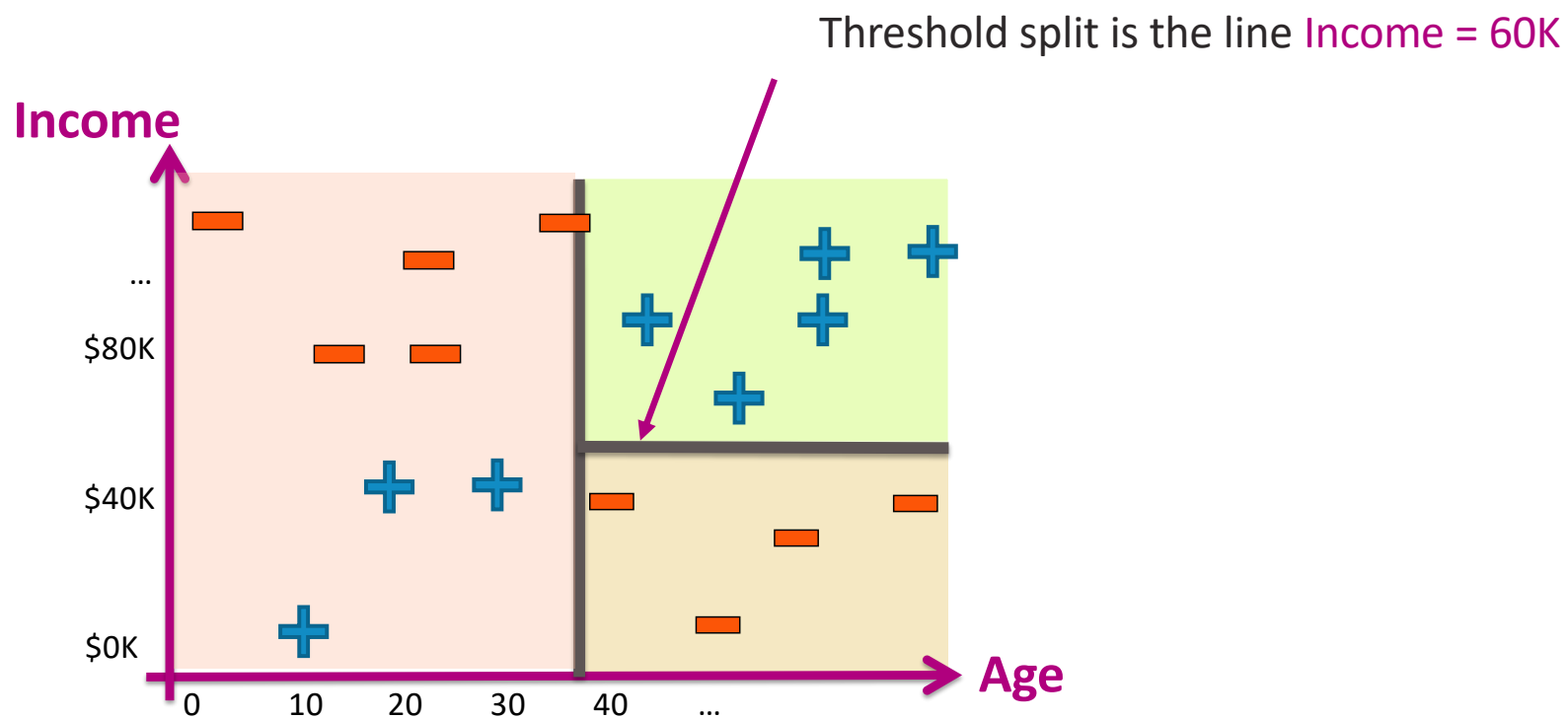
Visualizing the threshold split



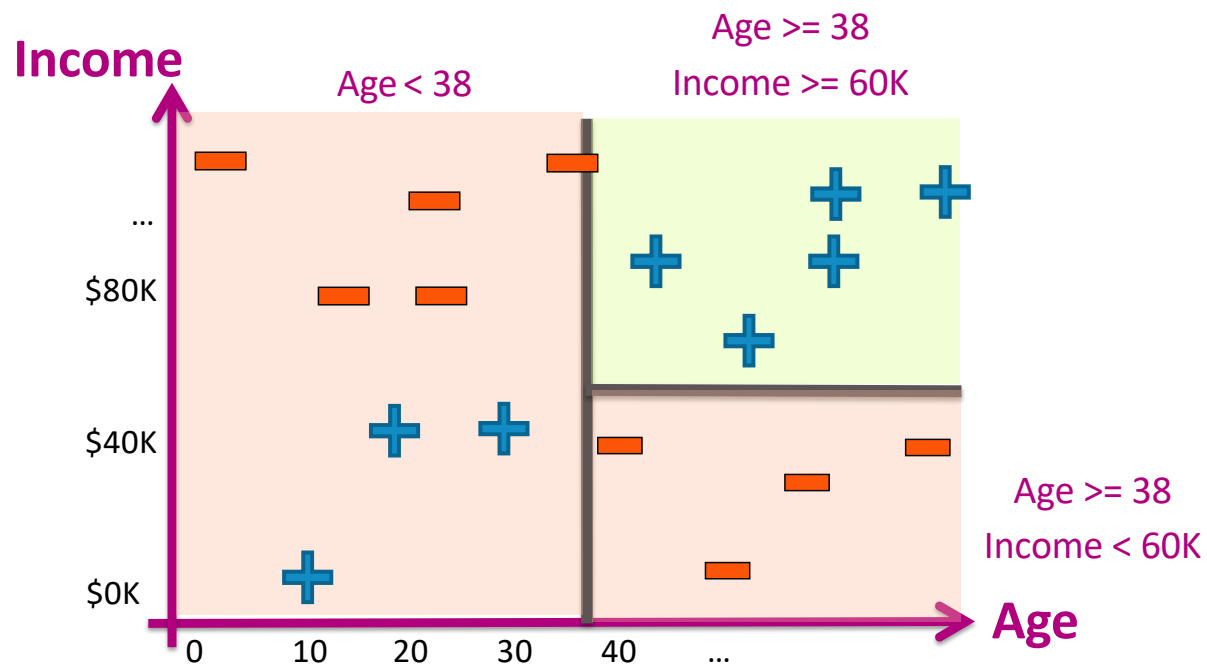
Split on Age ≥ 38



Depth 2: Split on Income $\geq$ \$60K



Each split partitions the 2-D space

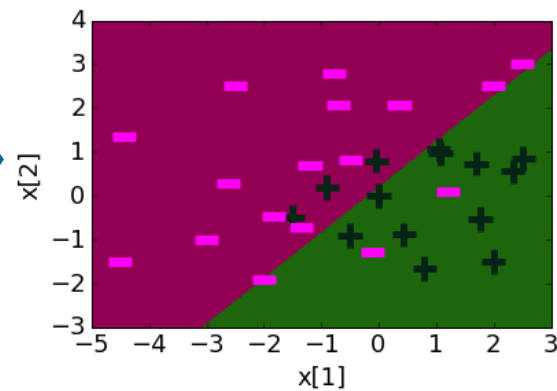
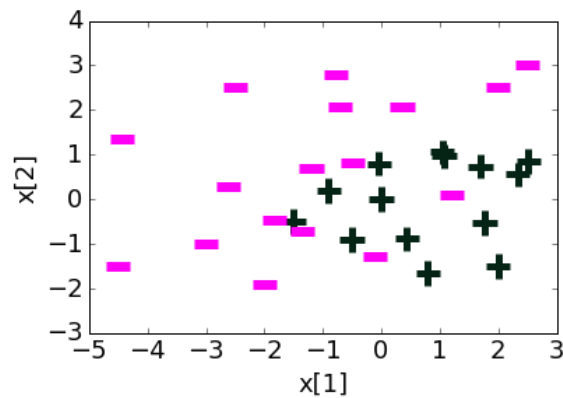


Decision trees vs logistic regression:

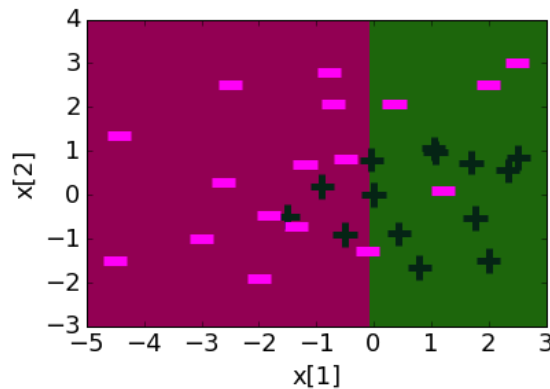
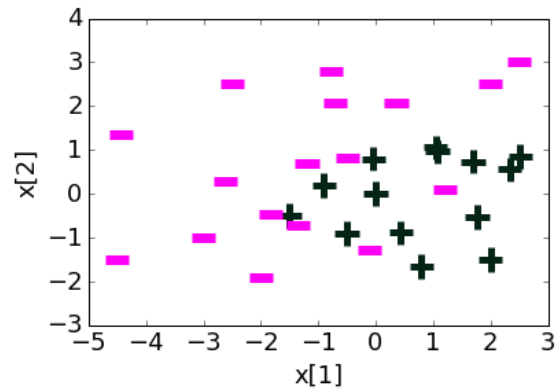
Example

Logistic regression

Feature	Value	Weight Learned
$h_0(x)$	1	0.22
$h_1(x)$	$x[1]$	1.12
$h_2(x)$	$x[2]$	-1.07

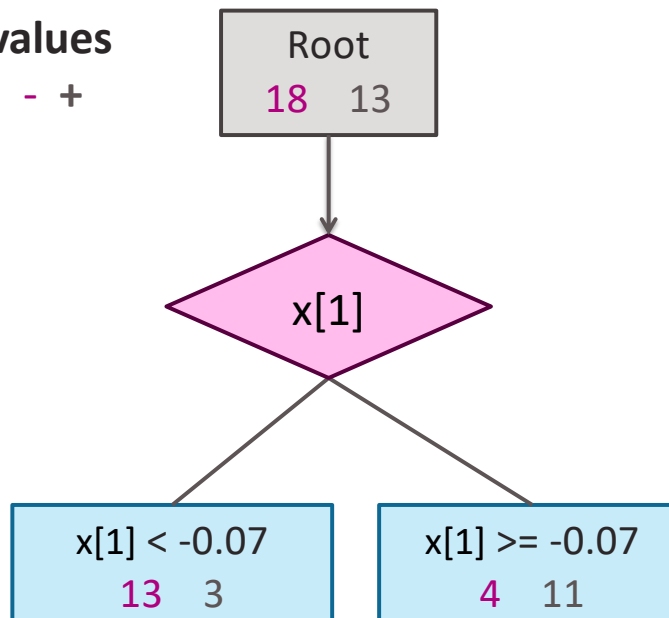


Depth 1: Split on $x[1]$

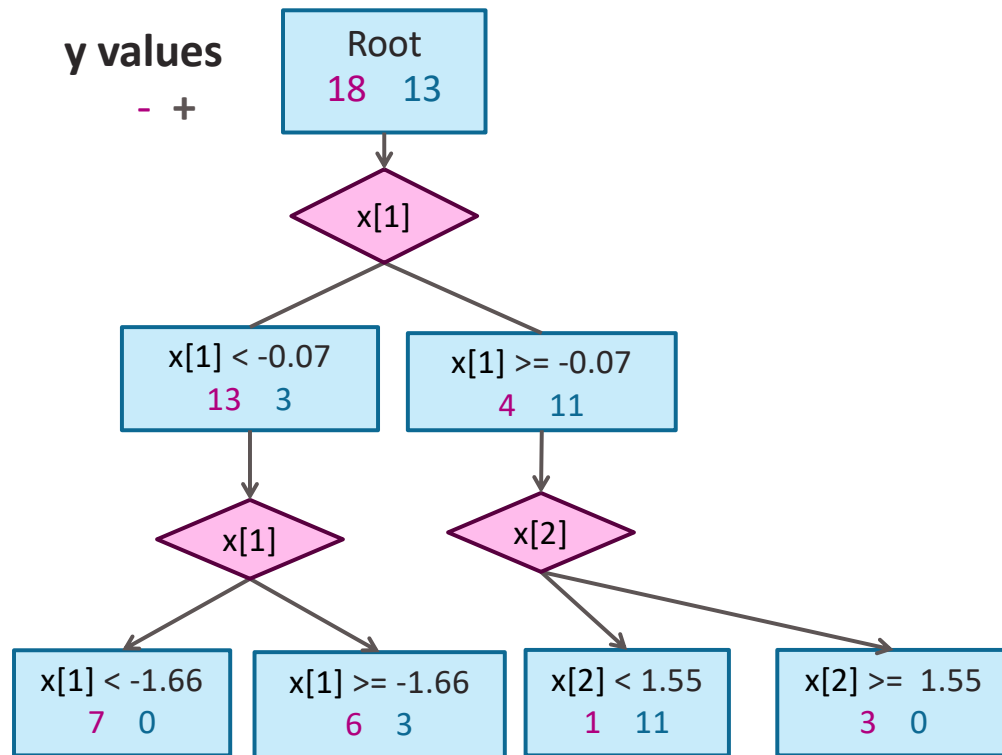
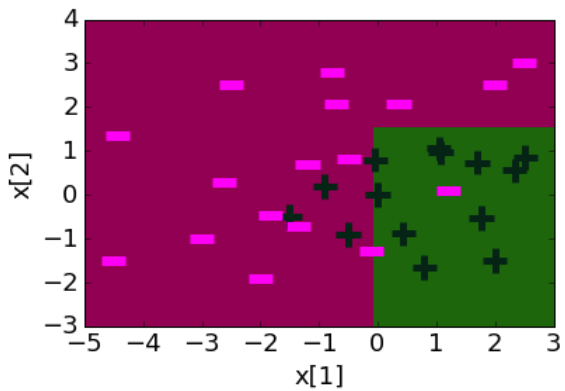
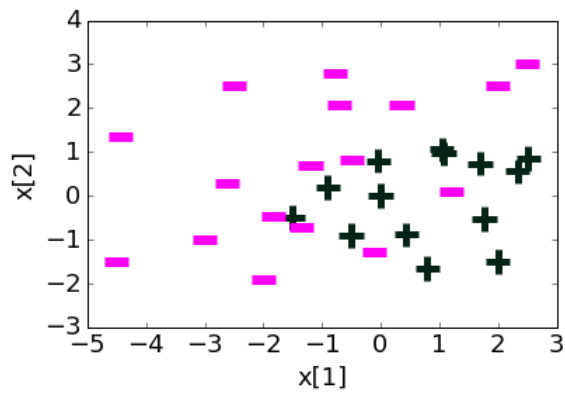


y values

- +

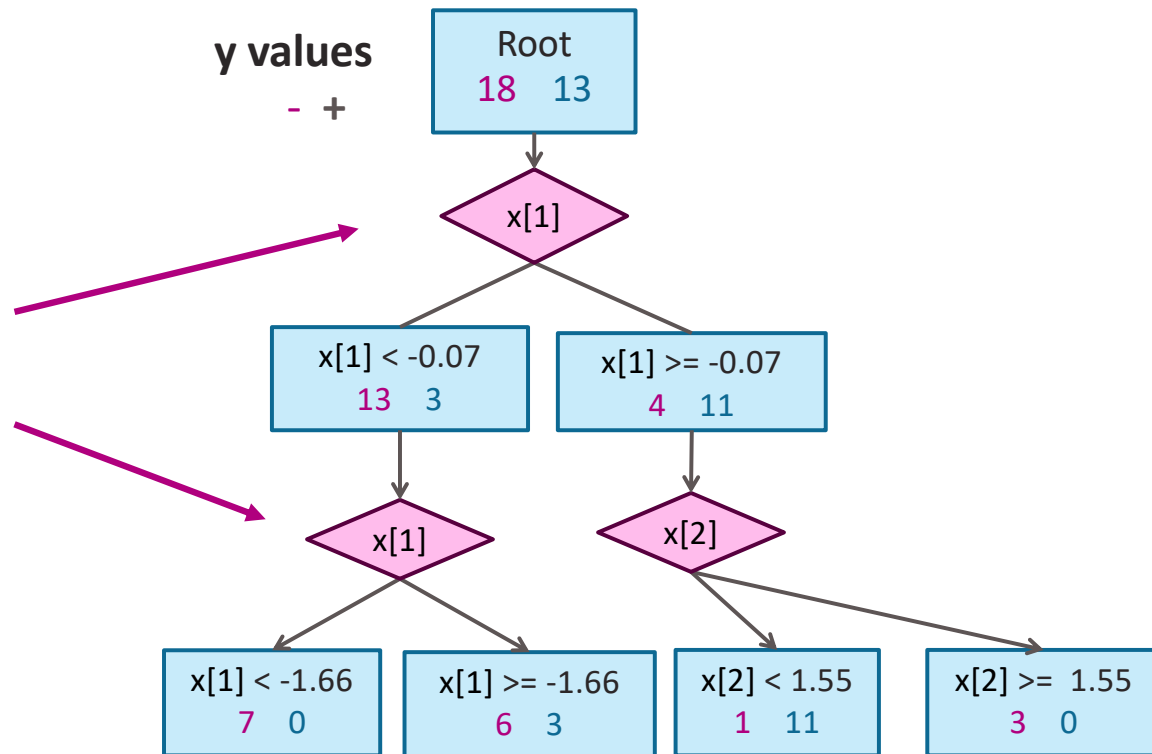


Depth 2

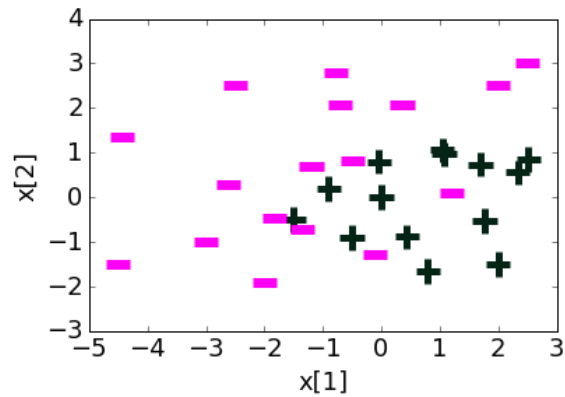


Threshold split caveat

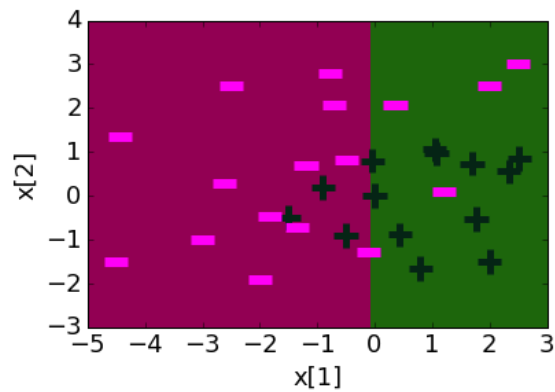
For threshold splits,
same feature can be
used multiple times



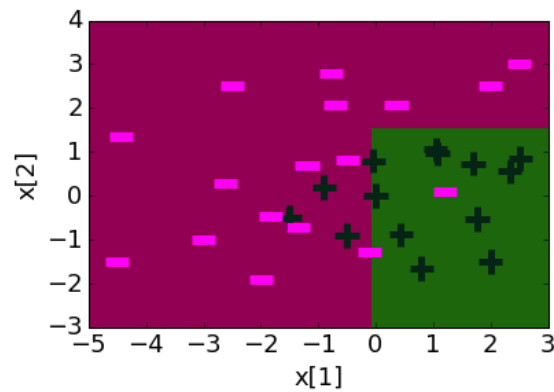
Decision boundaries



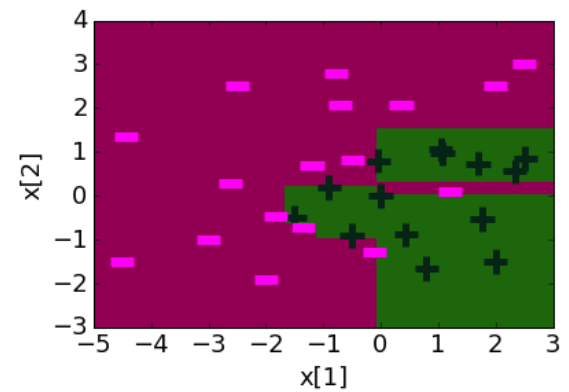
Depth 1



Depth 2

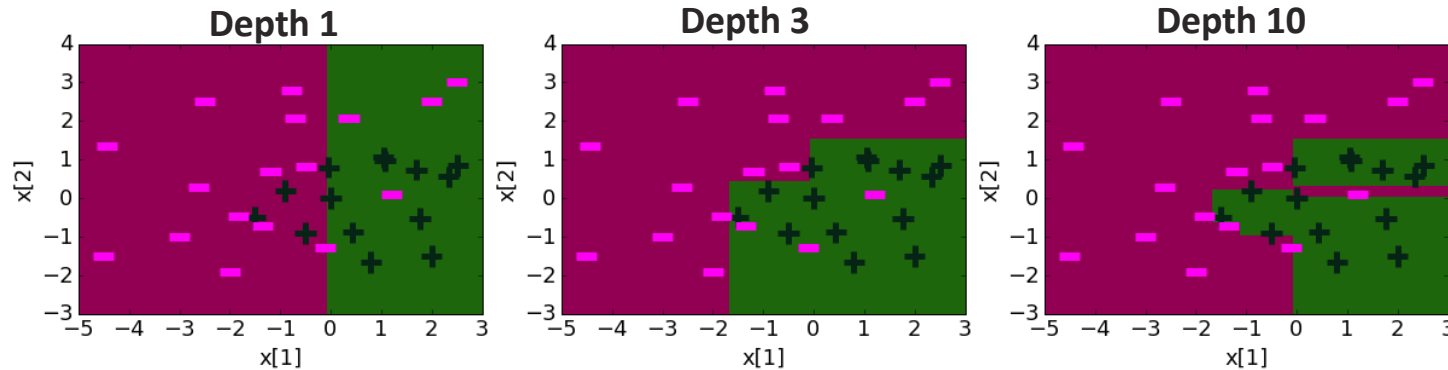


Depth 10

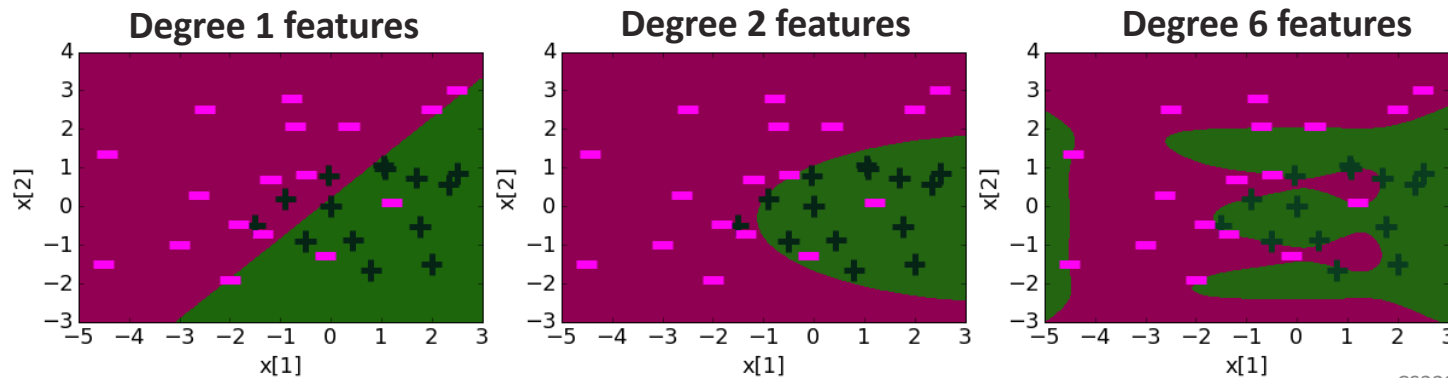


Comparing decision boundaries

Decision Tree



Logistic Regression



Summary of decision trees

What you can do now

- Define a decision tree classifier
- Interpret the output of a decision trees
- Learn a decision tree classifier using greedy algorithm
- Traverse a decision tree to make predictions
 - Majority class predictions
- Tackle continuous and discrete features

Outline

- **Decision Trees Recap**
- Ensemble Methods: Intro
- Model Averaging
- Bagging
- Random Forests
- Boosting
- Gradient boosting

Decision Trees Recap

Pros

- Can handle large datasets
- Can handle mixed predictors (continuous, discrete, qualitative)
- Can ignore redundant variables
- Can easily handle missing data
- Easy to interpret if small

Cons

- Prediction performance is poor
- Does not generalize well
- Large trees are hard to interpret

Outline

- Decision Trees Recap
- **Ensemble Methods: Intro**
- Model Averaging
- Bagging
- Random Forests
- Boosting
- Gradient boosting

Ensemble Methods: Intro

- Methods to improve the performance of weak learners
- Weak learners (e.g., classification trees) don't perform that well
- What do we do??
- **Wisdom of the crowds!**

Ensemble Methods: Intro

- **Wisdom of the crowds!**
- Shift responsibility from 1 weak learner to an “ensemble” of such weak learners
- Set of weak learners are combined to form a strong learner with better performance than any of them individually

Ensemble Methods: Intro

- A single decision tree often produces noisy / weak classifiers
- They DON'T generalize well
- But they are super fast, adaptive and robust!
- **Solution: Let's learn multiple trees!**
- How to ensure they don't all just learn the same thing??
- ~~TRIVIAL Solution~~

Outline

- Decision Trees Recap
- Ensemble Methods: Intro
- **Bagging**
- Random Forests
- Boosting
- Gradient boosting

Bagging

- Bagging (Breiman, 1996)
- **Bootstrap Aggregating**: to ensure lower variance
- **Bootstrap sampling**: get different splits / subsets of the data
- **Aggregating**: majority voting or averaging

Bagging

- Averages a given procedure over many samples to reduce its variance
- Multiple realizations of the data (via multiple samples) →
 - calculate predictions multiple times →
 - average the predictions →
 - more certain estimations (lesser variance)

Bagging

- Let $f(x)$ be the classifier and let b be a sample set from data

$$\hat{f}_{agg}(x) = \frac{1}{B} \sum_{b=1}^B f_b(x)$$

Or

$$\hat{f}_{agg}(x) = \text{Majority Vote } \{f_b(x)\}_{b=1}^B$$

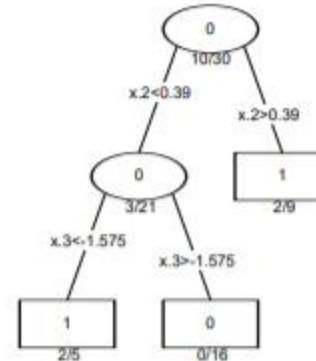
- Independent of type of classifier

Bagging

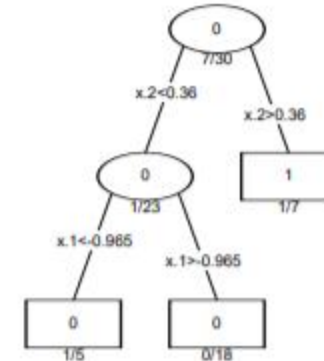
- Bootstrap sampling:
- Collect B ($\cong 100$) subsets by sampling with replacement from training data
- Construct B trees (one classifier for one subset)
- Aggregate them using aggregator of your choice
- Parallelizable

Bagging

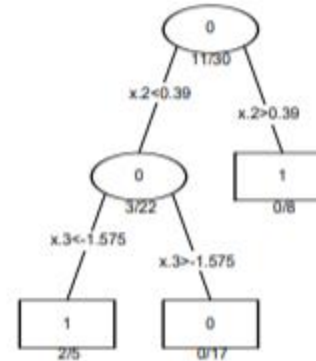
Original Tree



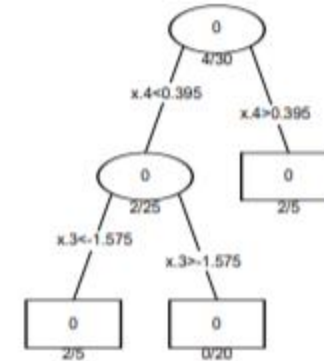
Bootstrap Tree 1



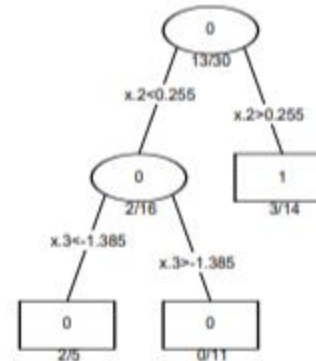
Bootstrap Tree 2



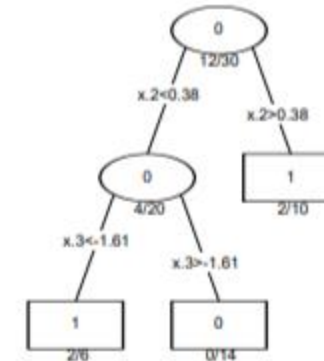
Bootstrap Tree 3



Bootstrap Tree 4

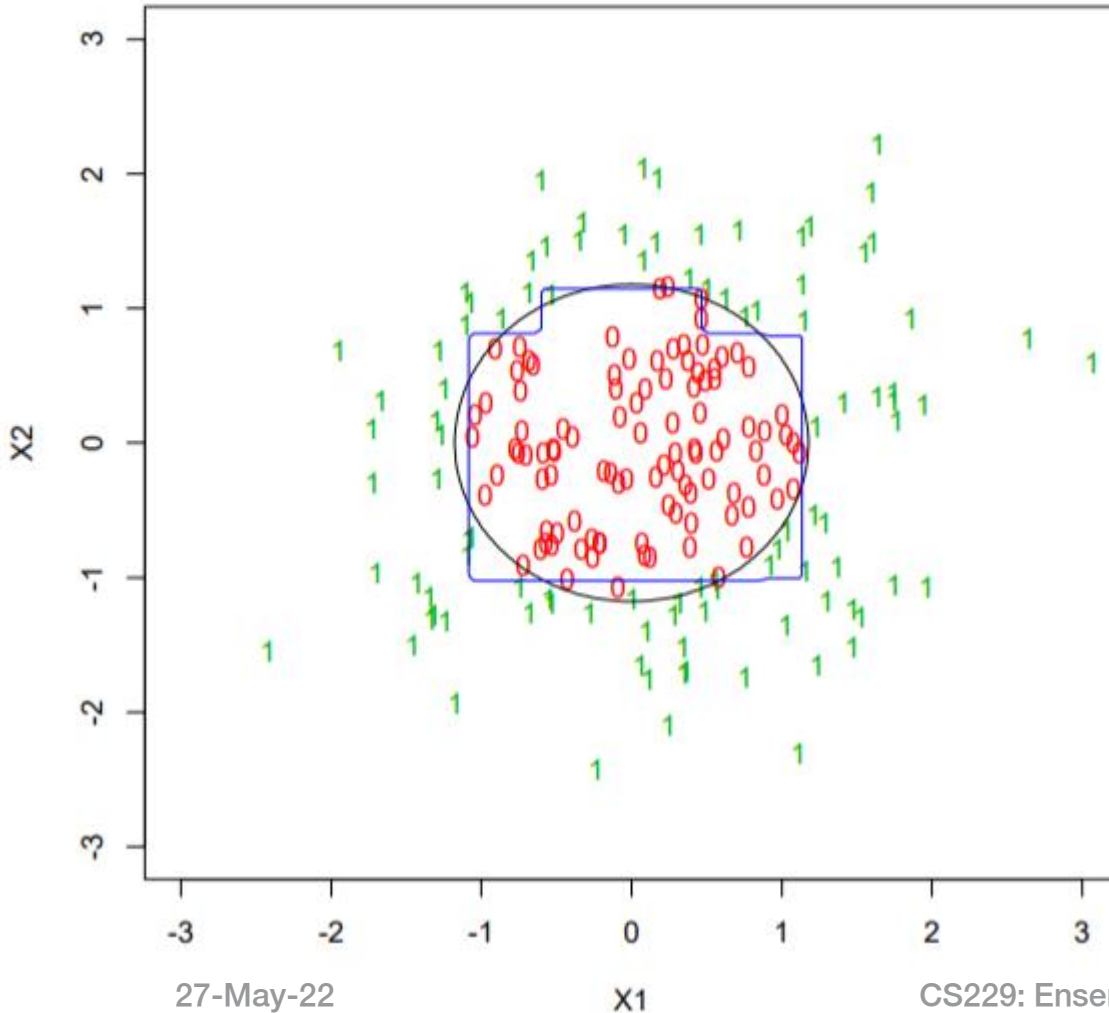


Bootstrap Tree 5

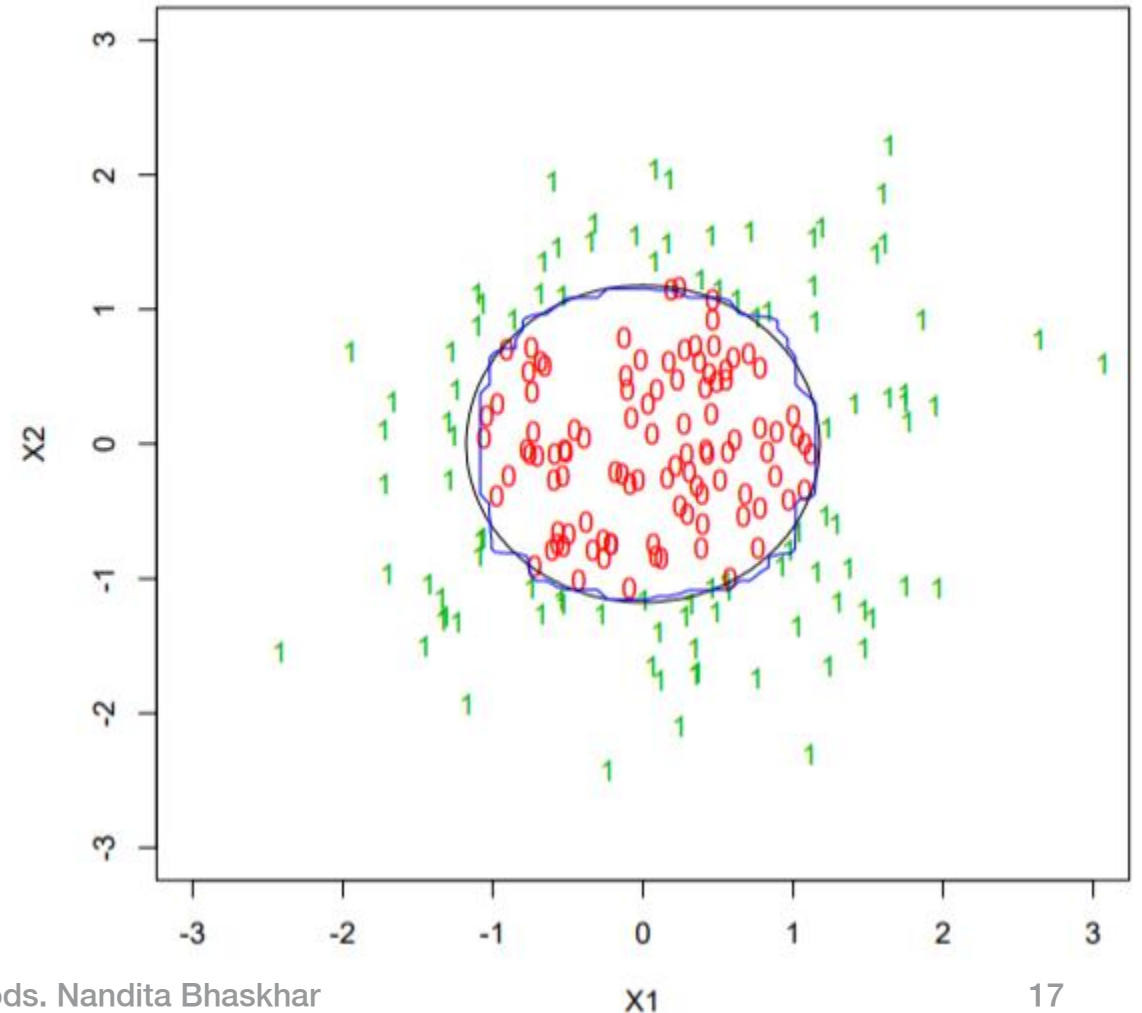


Bagging

Error Rate: 0.073



Error Rate: 0.032



Bagging

- What about cross validation?
- Each bootstrap sample set uses only a subset of the data
- Unused samples: out-of-bag samples (OOB)
- Calculate overall error rate on out-of-bag samples for all bootstraps

Bagging

- Reduces overfitting (i.e., variance)
- Can work with any type of classifier (here focus on trees)
- Easy to parallelize
- But loses on interpretability to single decision tree

Outline

- Decision Trees Recap
- Ensemble Methods: Intro
- Bagging
- **Random Forests**
- Boosting
- Gradient boosting

Random Forests

Issues with Bagging:

- Expectation of bagged trees is equal to expectation of individual trees

$$\mathbf{E} [\hat{f}_{agg}(x)] = \mathbf{E} [f_b(x)]$$

- Bias of bagged trees is the same as that of individual trees
- Each tree is **identically distributed** (i.d. not i.i.d). Bagged trees are **correlated!**

Random Forests

Issues with Bagging:

- Averaging B **i.i.d.** variables scales their variance σ^2 to σ^2/B
- But averaging B **i.d.** variables with pairwise correlations ρ and variance σ^2 gives their final variance to be

$$\rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$$

- Only the 2nd term reduces on bagging, but the first term remains

Random Forests

- How to **decorrelate** the trees generated for bagging?
- We want to generate B i.i.d. trees such that their bias is the same, but variance reduces
- **Ideas:**
 - We can restrict how many times a feature can be used
 - We only allow a certain number of features
 - Etc..

Random Forests

- **Ideas:**

- We can restrict how many times a feature can be used
- We only allow a certain number of features
- Etc..

- **Bias changes for the above ideas 😞**

- Instead, choose only subset of features for each bag
- Decorrelated trees when you **randomly** select the subset

Random Forests

- As in bagging, choose B bootstrapped splits (or bags)
- For each split in the B trees, consider only k features from the full feature set m
- $k = m \rightarrow$ same as Bagging
- $k < m \rightarrow$ Random Forests
- OOB error rate can be used to fit RF in one sequence with cross validation done along the way

Random Forests

- Works great in practice. k to be treated as a hyperparameter

Issues:

- When you have large number of features, yet very small number of **relevant** features
- Prob(selecting the relevant feature in k) is very small

Outline

- Decision Trees Recap
- Ensemble Methods: Intro
- Bagging
- Random Forests
- **Boosting**
- Gradient boosting

Boosting

- Boosting does not involve bootstrap sampling
- Trees are grown sequentially: each tree is grown using information from previously grown trees
- Like bagging, boosting involves combining many decision trees, $f_1, \dots, f_B$

Boosting

- **AdaBoost:**
- Weighted observations
- Put more weight on difficult to classify instances and less on those already handled well
- New weak learners are added sequentially that focus their training on the more difficult patterns

Outline

- Decision Trees Recap
- Ensemble Methods: Intro
- Bagging
- Random Forests
- Boosting
- **Gradient boosting**

Gradient Boosting

Generalization: AdaBoost, Adaptive Reweighting & Combining

Three elements –

- A loss function to be optimized
- A weak learner to make predictions (decision trees)
- An additive model to add weak learners to minimize the loss function

Gradient Boosting

Loss Function:

- Any differentiable function
- Most standard loss functions
 - L2 loss for regression
 - Log loss for classification

Gradient Boosting

Weak Learners:

- Decision trees (regression trees) learnt greedily
- Constrain the trees to ensure they remain weak
 - Number of layers, leaves, nodes, splits, etc

Gradient Boosting

Additive Model:

- Add trees one at a time (existing trees are not changed)
- Functional gradient descent to minimize loss when adding trees
 - calculate the loss
 - add the tree to the model that reduces the loss (i.e., follow the gradient)
 - parameterize the tree, then modify the parameters of the tree and move in the right direction by reducing the residual loss.

Gradient Boosting

Given the current model,

- We fit a decision tree to the **residuals** from the model
- Response variable now is the residuals
- We then add this new decision tree into the fitted function in order to update the residuals
- The learning rate must be controlled

Gradient Boosting

Tunable Parameters:

- **Number of trees (B):** Boosting can overfit unlike Bagging / RFs. Use cross-validation!
- **Shrinkage parameter (λ):** small positive number that sets the learning rate
- **Number of splits in each tree (d):** Usually just choose $d = 1$, i.e., tree stumps work well

Gradient Boosting

Variants:

- Varying the tree constraints
- Weighting each tree to the additive sum using a learning rate (shrinkage)
- Sampling strategies: stochastic gradient boosting
- Regularization: L1 / L2
- Successful: XGBoost

Thank you!

General tips for projects:

- Use tree-based methods + ensembling as baselines when dealing with:
 - Categorical data
 - Mixed data types
 - Missing data

CS229 Section: Midterm Review

Nandita Bhaskhar

Content from past CS229 teams and ML Cheatsheets from Shervine & Afshine Amidi

May 6, 2022

Outline

- 1 Supervised Learning
- 2 Optimization
- 3 Linear Regression
- 4 Logistic Regression
- 5 Exponential Family
- 6 GLMs
- 7 Generative Algorithms
- 8 Kernels
- 9 NNs

Supervised Learning: Recap

- **Given:** a set of data points (or attributes) $\{x^{(1)}, x^{(2)}, \dots, x^{(m)}\}$ and their associated labels $\{y^{(1)}, y^{(2)}, \dots, y^{(m)}\}$
- **Dimensions:** x usually d -dimensional $\in \mathbb{R}^d$, y typically scalar
- **Goal:** build a model that predicts y from x for unseen x

Supervised Learning: Recap

Types of predictions

- y is continuous, real-valued: Regression
- Example: Linear regression
- y is discrete classes: Classification
- Example: Logistic regression, SVM, Naive Bayes

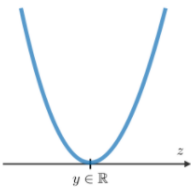
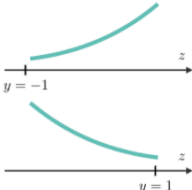
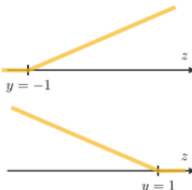
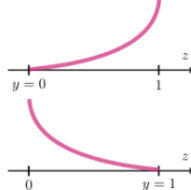
Supervised Learning: Recap

Types of models

- **Discriminative**
 - Directly estimate $p(y|x)$ by learning decision boundary
 - Example: Logistic regression, SVM
- **Generative**
 - Models the joint distribution $p(x, y)$
 - Estimate $p(x|y)$ and infer $p(y|x)$ from it
 - Can generate new samples
 - Example: GDA, Naive Bayes

Notations and Concepts

- **Hypothesis:** Denoted by h_θ . Given an input $x^{(i)}$, predicted output is $h_\theta(x^{(i)})$
- **Loss Function:** Function $L(z, y) : \mathbb{R} \times \mathbb{Y} \mapsto \mathbb{R}$ computes how different the predicted value z and the ground truth label are

Least squared error	Logistic loss	Hinge loss	Cross-entropy
$\frac{1}{2}(y - z)^2$	$\log(1 + \exp(-yz))$	$\max(0, 1 - yz)$	$-\left[y \log(z) + (1 - y) \log(1 - z)\right]$
			
Linear regression	Logistic regression	SVM	Neural Network

Notations and Concepts

- **Cost function:** Function J taking model parameters θ as input and giving a score to reflect how badly the model performs. Sum of loss over all predictions

$$J(\theta) = \sum_{i=1}^m L(h_{\theta}(x^{(i)}), y^{(i)})$$

- **Likelihood:** Maximizing likelihood $L(\theta)$ corresponds to finding the "best" distribution of data given a set of parameters. We usually find the log likelihood $\ell(\theta) = \log L(\theta)$ and maximize it.

$$\theta^* = \operatorname{argmax}_{\theta} \ell(\theta)$$

Outline

- 1 Supervised Learning
- 2 Optimization**
- 3 Linear Regression
- 4 Logistic Regression
- 5 Exponential Family
- 6 GLMs
- 7 Generative Algorithms
- 8 Kernels
- 9 NNs

Optimization: Gradient Descent

- To find the optimal θ that minimizes the cost function $J(\theta)$, we can use gradient descent with a learning rate $\alpha \in \mathbb{R}$

$$\theta^{(t+1)} = \theta^{(t)} - \alpha \nabla_{\theta} J(\theta^{(t)})$$

Stochastic Gradient Descent

- In Stochastic gradient descent (SGD), we update the parameter based on **each** training example, whereas in batch gradient descent we update based on a batch of training examples.

Optimization: Newton's method

- Numerical method to estimate θ such that $J'(\theta)$ is 0
- We update θ as follows:

$$\theta^{(t+1)} = \theta^{(t)} - \frac{J'(\theta^{(t)})}{J''(\theta^{(t)})}$$

- For the multi-dimensional case:

$$\theta^{(t+1)} = \theta^{(t)} - \left[\nabla_{\theta}^2 J(\theta^{(t)}) \right]^{-1} \nabla_{\theta} J(\theta^{(t)})$$

Recap: Gradients and Hessians

- Gradient and Hessian (differentiable function $f : \mathbb{R}^d \mapsto \mathbb{R}$)

$$\nabla_x f = \left[\frac{\partial f}{\partial x_1} \quad \cdots \quad \frac{\partial f}{\partial x_d} \right]^T \in \mathbb{R}^d$$

$$\nabla_x^2 f = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_d} \\ \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_d \partial x_1} & \cdots & \frac{\partial^2 f}{\partial x_d^2} \end{bmatrix} \in \mathbb{R}^{d \times d}$$

Outline

- 1 Supervised Learning
- 2 Optimization
- 3 Linear Regression**
- 4 Logistic Regression
- 5 Exponential Family
- 6 GLMs
- 7 Generative Algorithms
- 8 Kernels
- 9 NNs

Linear Regression

- Model: $h_{\theta}(x) = \theta^T x$
- Training data: $\{(x^{(i)}, y^{(i)})\}_{i=1}^n, x^{(i)} \in \mathbb{R}^d$
- Loss: $J(\theta) = \frac{1}{2} \sum_{i=1}^n (h_{\theta}(x^{(i)}) - y^{(i)})^2$
- Update rule:

$$\theta^{(t+1)} = \theta^{(t)} - \alpha \sum_{i=1}^n (h_{\theta}(x^{(i)}) - y^{(i)}) x^{(i)}$$

Stochastic Gradient Descent (SGD)

Pick one data point $x^{(i)}$ and then update:

$$\theta^{(t+1)} = \theta^{(t)} - \alpha (h_{\theta}(x^{(i)}) - y^{(i)}) x^{(i)}$$

Solving Least Squares: Closed Form

- Loss in matrix form: $J(\theta) = \frac{1}{2} \|X\theta - y\|_2^2$, where $X \in \mathbb{R}^{n \times d}$, $y \in \mathbb{R}^n$
- Normal Equation (set gradient to 0):

$$X^T (X\theta^* - y) = 0$$

- Closed form solution:

$$\theta^* = (X^T X)^{-1} X^T y$$

Connection to Newton's Method

$$\theta^* = [\nabla_{\theta}^2 J]^{-1} \nabla_{\theta} J, \quad \text{when the gradient is evaluated at } \theta = 0$$

Newton's method is exact with only one step iteration if we started from $\theta^{(0)} = 0$.

Outline

- 1 Supervised Learning
- 2 Optimization
- 3 Linear Regression
- 4 Logistic Regression**
- 5 Exponential Family
- 6 GLMs
- 7 Generative Algorithms
- 8 Kernels
- 9 NNs

Logistic Regression

A binary classification model and $y^{(i)} \in \{0, 1\}$

- Assumed model:

$$p(y | x; \theta) = \begin{cases} g_{\theta}(x) & \text{if } y = 1 \\ 1 - g_{\theta}(x) & \text{if } y = 0 \end{cases}, \quad \text{where } g_{\theta}(x) = \frac{1}{1 + e^{-\theta^T x}}$$

- Log-likelihood function:

$$\begin{aligned} \ell(\theta) &= \sum_{i=1}^n \log p(y^{(i)} | x^{(i)}; \theta) \\ &= \sum_{i=1}^n \left[y^{(i)} \log g_{\theta}(x^{(i)}) + (1 - y^{(i)}) \log(1 - g_{\theta}(x^{(i)})) \right] \end{aligned}$$

- Find parameters through **maximizing log-likelihood**, $\operatorname{argmax}_{\theta} \ell(\theta)$ (in Pset1).

Sigmoid and Softmax

- **Sigmoid:** The sigmoid function (also known as logistic function) is given by:

$$g(z) = \frac{1}{1 + e^{-z}}$$

- **Softmax regression:** Also called as multi-class logistic regression, it generalizes logistic regression to multi-class cases

$$p(y = k|x; \theta) = \frac{\exp \theta_k^T x}{\sum_j \exp \theta_j^T x}$$

Outline

- 1 Supervised Learning
- 2 Optimization
- 3 Linear Regression
- 4 Logistic Regression
- 5 Exponential Family**
- 6 GLMs
- 7 Generative Algorithms
- 8 Kernels
- 9 NNs

Exponential Family

Definition

Probability distribution with **natural or canonical parameter** η , **sufficient statistic** $T(y)$ and a **log-partition** function $a(\eta)$ whose density (or mass function) can be written as

$$p(y; \eta) = b(y) \exp \left(\eta^T T(y) - a(\eta) \right)$$

- Oftentimes, $T(y) = y$
- In many cases, $\exp(-a(\eta))$ can be considered as a normalization term that makes the probabilities sum to one

Common Exponential Distributions

Bernoulli distribution:

$$p(y; \phi) = \phi^y (1 - \phi)^{1-y} = \exp \left(\left(\log \left(\frac{\phi}{1 - \phi} \right) \right) y + \log(1 - \phi) \right)$$

$$\implies b(y) = y, \quad T(y) = y, \quad \eta = \log \left(\frac{\phi}{1 - \phi} \right), \quad a(\eta) = \log(1 + e^\eta)$$

More examples:

Categorical distribution, Poisson distribution, Multivariate normal distribution, etc

Common Exponential Distributions

Distribution	η	$T(y)$	$a(\eta)$	$b(y)$
Bernoulli	$\log\left(\frac{\phi}{1-\phi}\right)$	y	$\log(1 + \exp(\eta))$	1
Gaussian	μ	y	$\frac{\eta^2}{2}$	$\frac{1}{\sqrt{2\pi}} \exp\left(-\frac{y^2}{2}\right)$
Poisson	$\log(\lambda)$	y	e^η	$\frac{1}{y!}$
Geometric	$\log(1 - \phi)$	y	$\log\left(\frac{e^\eta}{1-e^\eta}\right)$	1

Properties

- $\mathbb{E}[T(Y); \eta] = \nabla_{\eta} a(\eta)$
- $\text{Var}(T(Y); \eta) = \nabla_{\eta}^2 a(\eta)$

Non-exponential Family Distribution

Uniform distribution over interval $[a, b]$:

$$p(y; a, b) = \frac{1}{b - a} \cdot 1_{\{a \leq y \leq b\}}$$

Reason: $b(y)$ cannot depend on parameter η .

Outline

- 1 Supervised Learning
- 2 Optimization
- 3 Linear Regression
- 4 Logistic Regression
- 5 Exponential Family
- 6 GLMs**
- 7 Generative Algorithms
- 8 Kernels
- 9 NNs

Generalized Linear Model (GLM)

Generalized Linear Models (GLM) aim at predicting a random variable y as a function of x and rely on the following components:

Assumed model:

$$p(y \mid x; \theta) \sim \text{ExponentialFamily}(\eta)$$

- $\eta = \theta^T x$
- Predictor: $h(x) = \mathbb{E}[T(Y); \eta] = \nabla_{\eta} a(\eta)$.
- Fitting through maximum likelihood:

$$\max_{\theta} \ell(\theta) = \max_{\theta} \sum_{i=1}^n p(y^{(i)} \mid x^{(i)}; \eta)$$

Generalized Linear Model (GLM)

Examples

- GLM under Bernoulli distribution: Logistic regression
- GLM under Poisson distribution: Poisson regression (in Pset1)
- GLM under Normal distribution: Linear regression
- GLM under Categorical distribution: Softmax regression

Outline

- 1 Supervised Learning
- 2 Optimization
- 3 Linear Regression
- 4 Logistic Regression
- 5 Exponential Family
- 6 GLMs
- 7 Generative Algorithms**
- 8 Kernels
- 9 NNs

Gaussian Discriminant Analysis (GDA)

Generative Algorithm for Classification

- Learn $p(x | y)$ and $p(y)$
- Classify through Bayes rule: $\operatorname{argmax}_y p(y | x) = \operatorname{argmax}_y p(x | y) p(y)$

GDA Formulation

- Assume $p(x | y) \sim \mathcal{N}(\mu_y, \Sigma)$ for some $\mu_y \in \mathbb{R}^d$ and $\Sigma \in \mathbb{R}^{d \times d}$
- Estimate μ_y , Σ and $p(y)$ through maximum likelihood, which is

$$\operatorname{argmax} \sum_{i=1}^n \left[\log p(x^{(i)} | y^{(i)}) + \log p(y^{(i)}) \right]$$

$$p(y) = \frac{\sum_{i=1}^n 1_{\{y^{(i)}=y\}}}{n}, \mu_y = \frac{\sum_{i=1}^n 1_{\{y^{(i)}=y\}} x^{(i)}}{\sum_{i=1}^n 1_{\{y^{(i)}=y\}}}, \Sigma = \frac{1}{n} \sum_{i=1}^n (x^{(i)} - \mu_{y^{(i)}})(x^{(i)} - \mu_{y^{(i)}})^T$$

Naive Bayes

Formulation

- Assume $p(x | y) = \prod_{j=1}^d p(x_j | y)$
- Estimate $p(x_j | y)$ and $p(y)$ through maximum likelihood, which gives

$$p(x_j | y) = \frac{\sum_{i=1}^n 1_{\{x_j^{(i)}=x_j, y^{(i)}=y\}}}{\sum_{i=1}^n 1_{\{y^{(i)}=y\}}}, \quad p(y) = \frac{\sum_{i=1}^n 1_{\{y^{(i)}=y\}}}{n}$$

Laplace Smoothing

Assume x_j takes value in $\{1, 2, \dots, k\}$, the corresponding modified estimator is

$$p(x_j | y) = \frac{1 + \sum_{i=1}^n 1_{\{x_j^{(i)}=x_j, y^{(i)}=y\}}}{k + \sum_{i=1}^n 1_{\{y^{(i)}=y\}}}$$

Outline

- 1 Supervised Learning
- 2 Optimization
- 3 Linear Regression
- 4 Logistic Regression
- 5 Exponential Family
- 6 GLMs
- 7 Generative Algorithms
- 8 Kernels**
- 9 NNs

Kernel

- Core idea: reparametrize parameter θ as a linear combination of featurized vectors
- Feature map: $\phi : \mathbb{R}^d \mapsto \mathbb{R}^p$
- Fitting linear model with gradient descent gives us

$$\theta = \sum_{i=1}^n \beta_i \phi(x^{(i)})$$

- Predict a new example z :

$$h_{\theta}(z) = \sum_{i=1}^n \beta_i \phi(x^{(i)})^T \phi(z) = \sum_{i=1}^n \beta_i K(x^{(i)}, z)$$

- It brings nonlinearity without much sacrifice in efficiency as long as $K(\cdot, \cdot)$ can be computed efficiently

Kernel

- Given a feature mapping ϕ , we define the kernel K as follows:

$$K(x, z) = \phi(x)^T \phi(z)$$

- "Kernel trick" to compute the cost function using the kernel because we actually don't need to know the explicit mapping ϕ , which is often very complicated
- Instead, only the values $K(x, z)$ are needed
- Suppose $K(x^{(i)}, x^{(j)}) = K_{ij}$
- If $K = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}$ then is K a valid kernel function?
- If $K = \begin{bmatrix} 3 & 5 \\ 5 & 3 \end{bmatrix}$ then is K a valid kernel function?

Kernel

Theorem

$K(x, z)$ is a valid kernel if and only if for any set of $\{x^{(1)}, \dots, x^{(n)}\}$, its Gram matrix, defined as

$$G = \begin{bmatrix} K(x^{(1)}, x^{(1)}) & \dots & K(x^{(1)}, x^{(n)}) \\ \vdots & \ddots & \vdots \\ K(x^{(n)}, x^{(1)}) & \dots & K(x^{(n)}, x^{(n)}) \end{bmatrix} \in \mathbb{R}^{n \times n}$$

is positive semi-definite.

Examples

- Polynomial kernels: $K(x, z) = (x^T z + c)^d$, $\forall c \geq 0$ and $d \in \mathbb{N}$
- Gaussian kernels: $K(x, z) = \exp\left(-\frac{\|x-z\|_2^2}{2\sigma^2}\right)$, $\forall \sigma^2 > 0$